

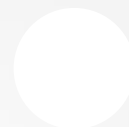
网络流与匹配

《算法设计与分析》

信息科学与技术学院 张德富



02



最大流问题求解



FordFulkerson方法

FordFulkerson方法（经典方法）

- 基本思想是从任何一个可行流开始，沿增广路径对流进行增广，然后不断重复此过程，直到网络中不存在增广路径为止。

问题的关键

- 如何有效地找到增广路径，并保证算法在有限次增广后一定终止

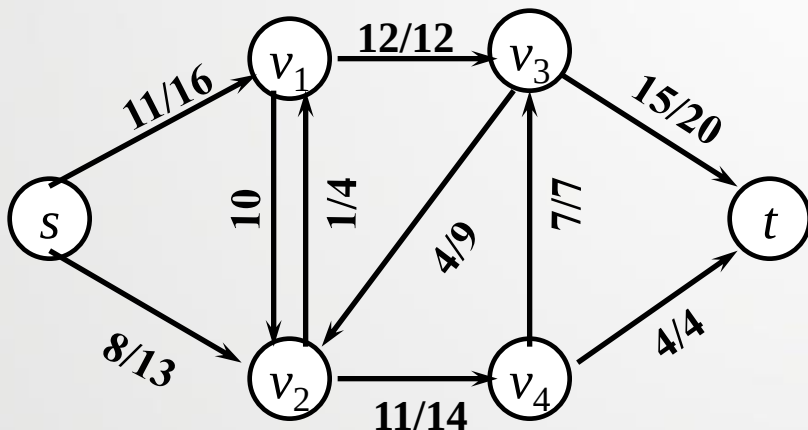
这个方法包含三个重要的概念：

- 剩余网络
- 增广路径
- 网络的割

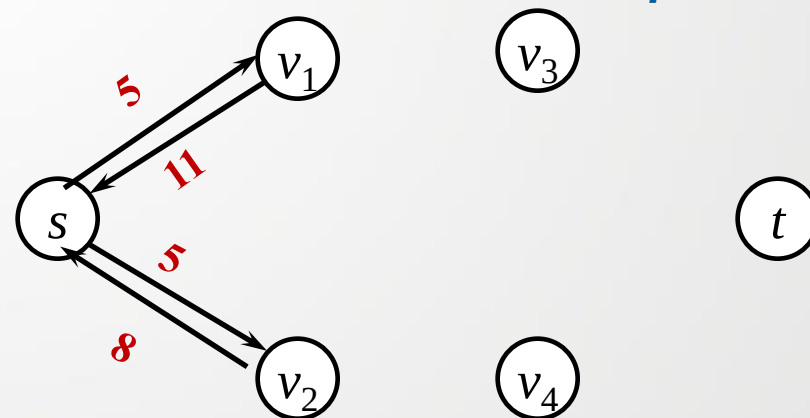
>>> 剩余网络(residual network)

- **定义9.3** 给定一个网络 G , 其流为 f , 容量函数为 c 。关于流 f 的剩余容量函数 c_f 定义如下: 对任意一对顶点 $u, v \in V$, $c_f(u, v) = c(u, v) - f(u, v)$ 。流 f 的剩余网络是一个有向图 $G_f = (V, E_f)$, 其容量函数由 c_f 定义, 边集由 $E_f = \{(u, v) | c_f(u, v) > 0\}$ 确定。

Flow network $G = (V, E)$



residual network $G_f = (V, E_f)$

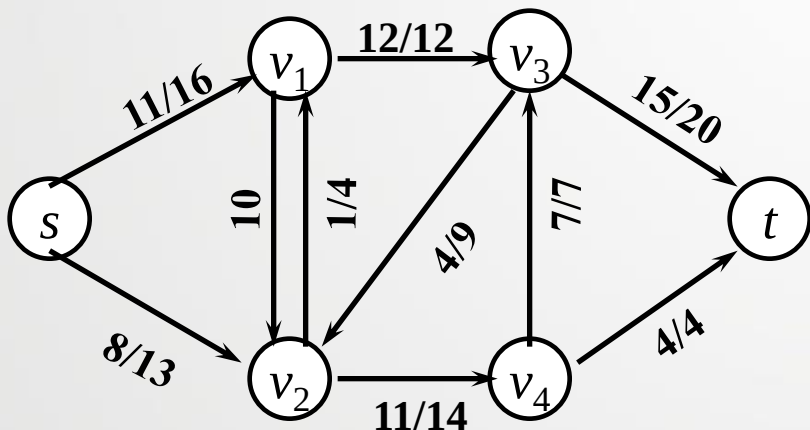


$$c_f(u, v) = c(u, v) - f(u, v)$$

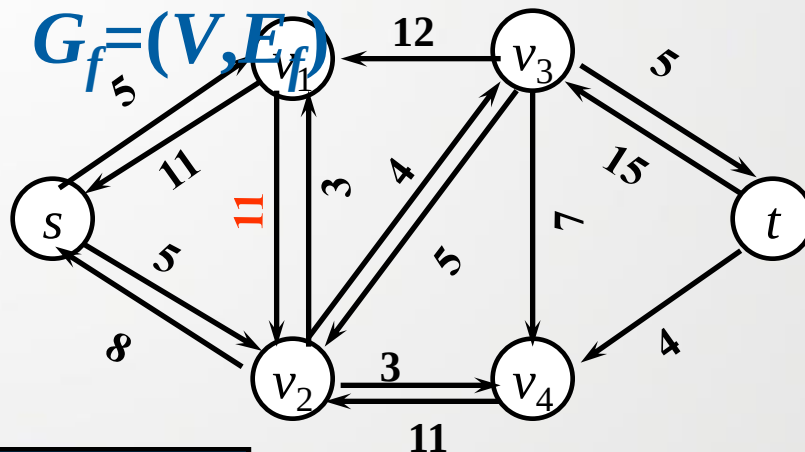
>>> 剩余网络(residual network)

- **定义9.3** 给定一个网络 G , 其流为 f , 容量函数为 c 。关于流 f 的剩余容量函数 c_f 定义如下: 对任意一对顶点 $u, v \in V$, $c_f(u, v) = c(u, v) - f(u, v)$ 。流 f 的剩余网络是一个有向图 $G_f = (V, E_f)$, 其容量函数由 c_f 定义, 边集由 $E_f = \{(u, v) | c_f(u, v) > 0\}$ 确定。

Flow network $G = (V, E)$



residual network



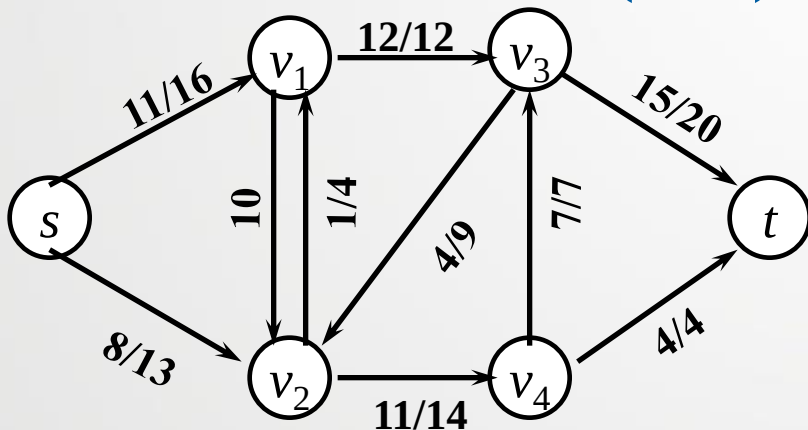
$$c_f(u, v) = c(u, v) - f(u, v)$$

增广路径(Augmenting path)

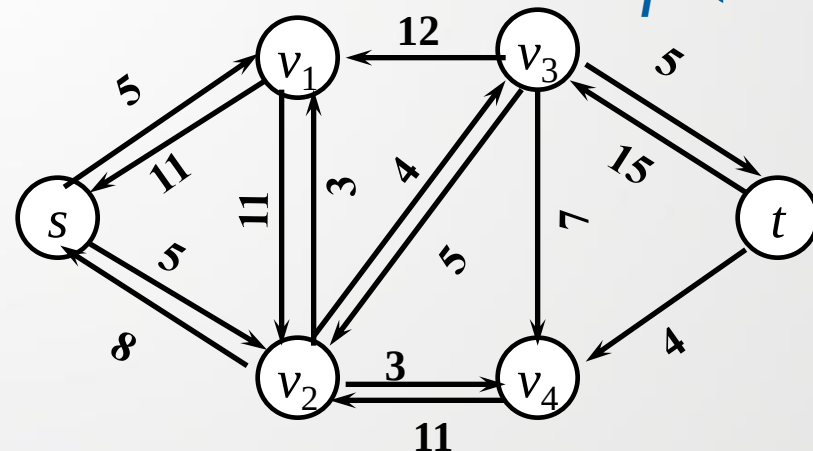
定义9.4 给定一个网络 G , 令 f_1 和 f_2 为 G 的流, 令 $(f_1 + f_2)(u,v) = f_1(u,v) + f_2(u,v)$, 则称 $f_1 + f_2$ 为流 f_1, f_2 的和。

定义9.5 给定一个网络 G , 令 f 为图 G 的一个流, 则称剩余网络 G_f 中从 s 到 t 的一条有向路径 p 为增广路径。

Flow network $G=(V,E)$



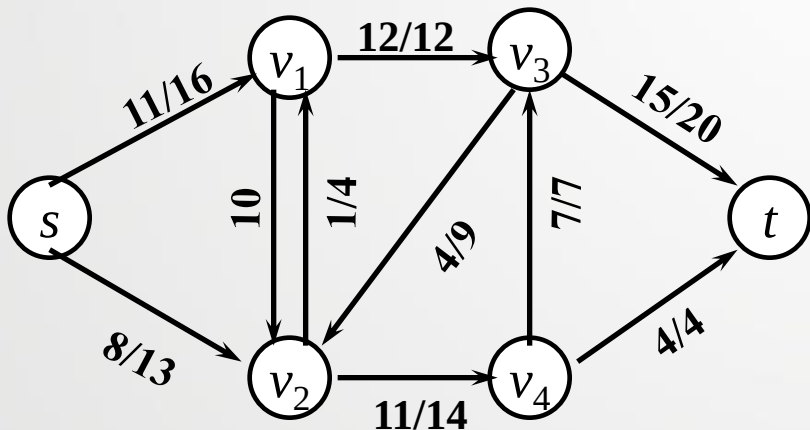
Residual network $G_f=(V,E_f)$



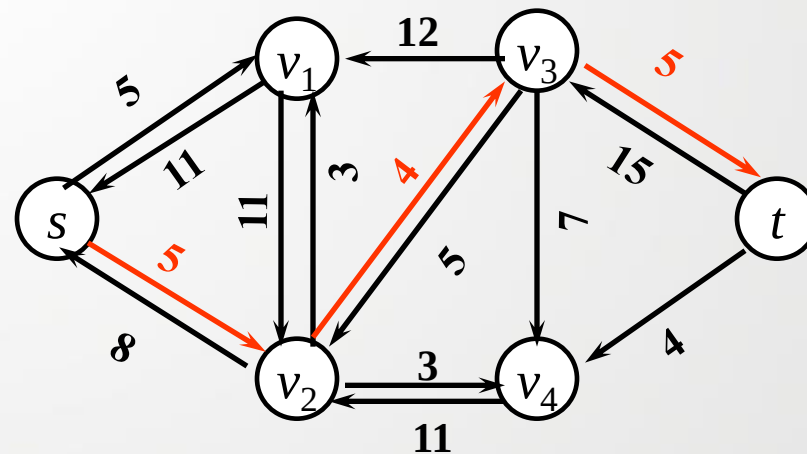
增广路径(Augmenting path)

定义9.5 给定一个网络 G , 令 f 为图 G 的一个流, 则称剩余网络 G_f 中从 s 到 t 的一条有向路径 p 为增广路径。

Flow network $G=(V,E)$



Residual network $G_f=(V,E_f)$



Augmenting path

>>> 增广路径(Augmenting path)

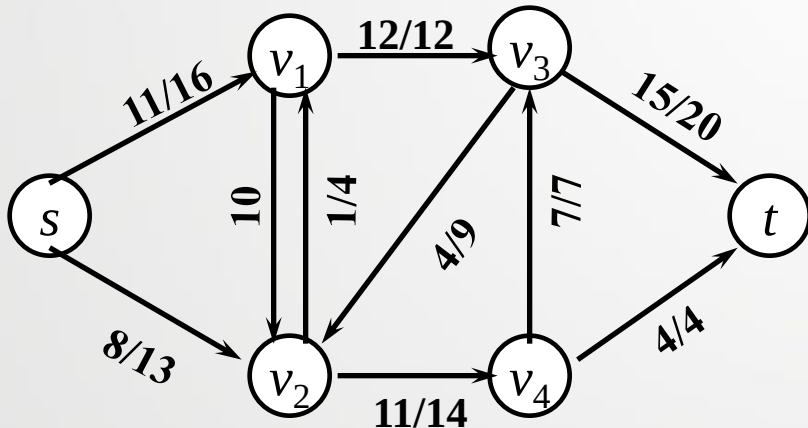
定义流 $f_p: V \times V \rightarrow \mathbb{R}$ 如下:

$$f_p(u,v) = \begin{cases} c_f(p) & \text{if } (u,v) \text{ is on } p \\ -c_f(p) & \text{if } (v,u) \text{ is on } p \\ 0 & \text{otherwise} \end{cases}$$

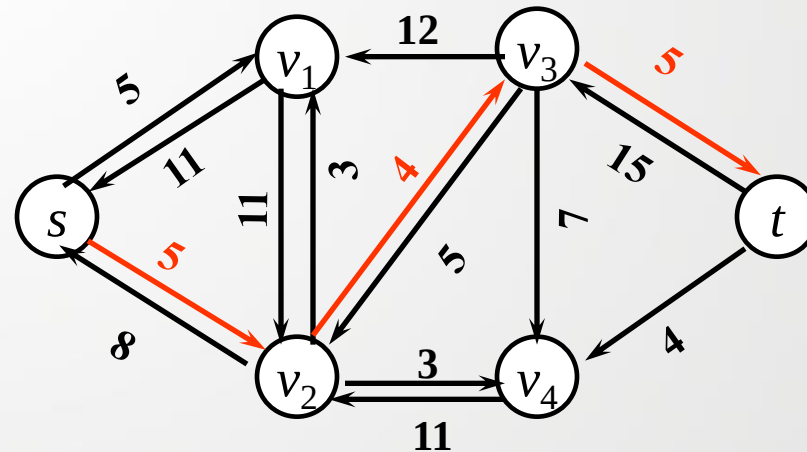
其中,

$$c_f(p) = \min\{c_f(u,v): (u,v) \text{ is on } p\}$$

Flow network $G = (V, E)$



Residual network $G_f = (V, E_f)$



G_f 中增广路径 p 上的流 f_p

增广路径(Augmenting path)

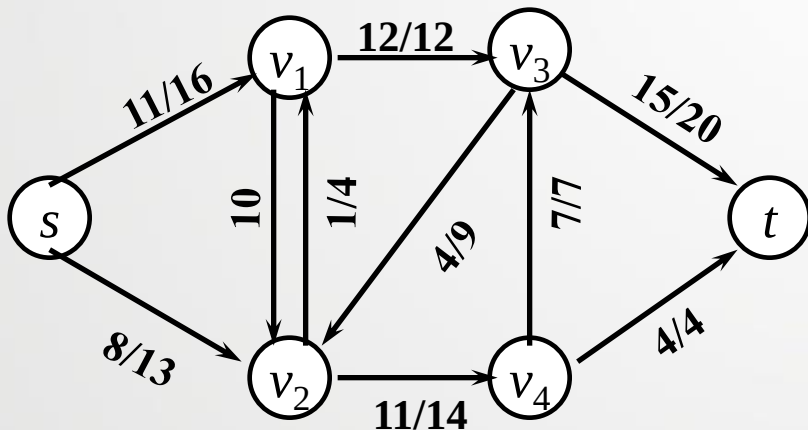
定义流 $f_p: V \times V \rightarrow \mathbb{R}$ 如下:

$$f_p(u,v) = \begin{cases} c_f(p) & \text{if } (u,v) \text{ is on } p \\ -c_f(p) & \text{if } (v,u) \text{ is on } p \\ 0 & \text{otherwise} \end{cases}$$

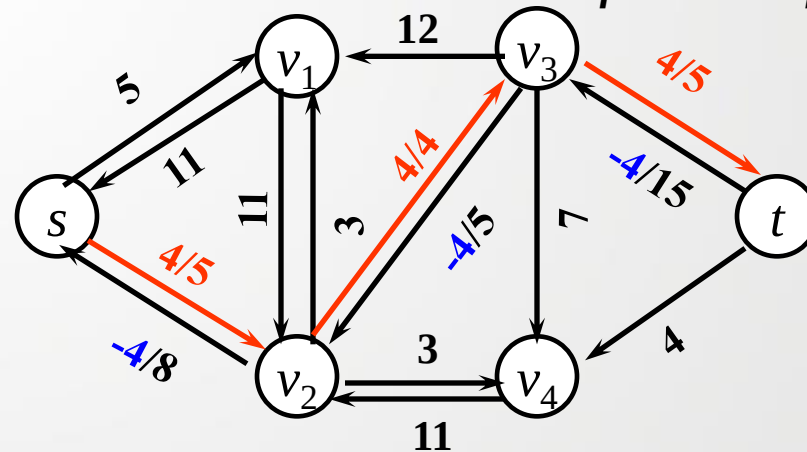
其中,

$$c_f(p) = \min\{c_f(u,v): (u,v) \text{ is on } p\}$$

Flow network $G = (V, E)$



Residual network $G_f = (V, E_f)$



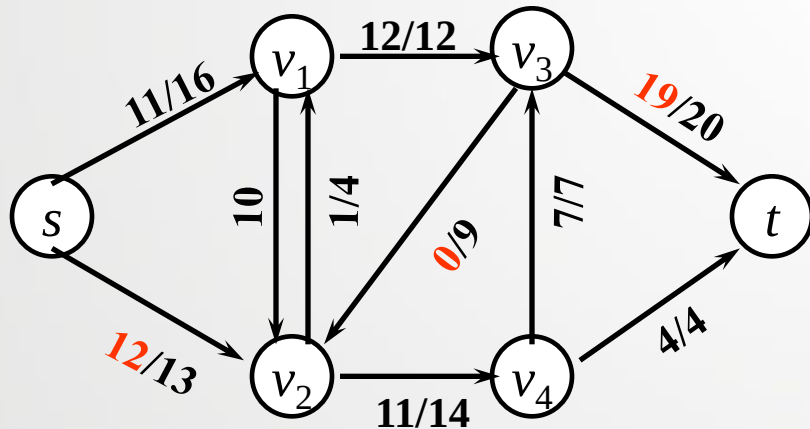
G_f 中增广路径 p 上的流 f_p

增广路径(Augmenting path)

定义流 $f_p: V \times V \rightarrow \mathbb{R}$ 如下:

$$f_p(u,v) = \begin{cases} c_f(p) & \text{if } (u,v) \text{ is on } p \\ -c_f(p) & \text{if } (v,u) \text{ is on } p \\ 0 & \text{otherwise} \end{cases}$$

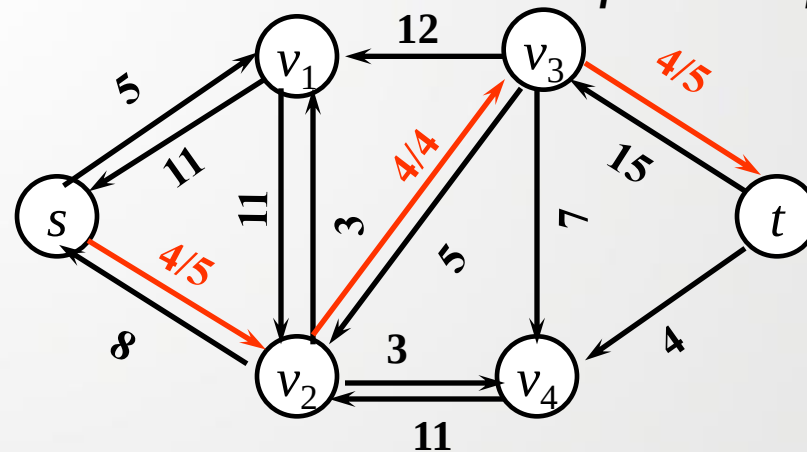
Flow network $G = (V, E)$



New flow:

$$f' = f + f_p: V \times V \rightarrow \mathbb{R}$$

Residual network $G_f = (V, E_f)$



G_f 中增广路径 p 上的流 f_p



New valid flow

引理

$$\hat{f} = f + f_p: V \times V \rightarrow \mathbb{R} \quad \text{in } G$$

容量约束:

任给 $u, v \in V$, 有 $\hat{f}(u, v) \leq c(u, v)$

Proof:

$$f_p(u, v) \leq c_f(u, v) = c(u, v) - f(u, v)$$

$$\Rightarrow (f + f_p)(u, v) = f(u, v) + f_p(u, v) \leq c(u, v)$$

$$f_p(u, v) = \begin{cases} c_f(p) & \text{if } (u, v) \text{ is on } p \\ -c_f(p) & \text{if } (v, u) \text{ is on } p \\ 0 & \text{otherwise} \end{cases}$$

$$c_f(p) = \min\{c_f(u, v): (u, v) \text{ is on } p\}$$

$$c_f(u, v) = c(u, v) - f(u, v)$$



New valid flow

引理

$$f' = f + f_p: V \times V \rightarrow \mathbb{R} \quad \text{in } G$$

反对称性质:

任给 $u, v \in V$, 有 $f'(u, v) = -f'(v, u)$

Proof:

$$\begin{aligned} (f + f_p)(u, v) &= f(u, v) + f_p(u, v) = -f(v, u) - f_p(v, u) \\ &= -(f(v, u) + f_p(v, u)) = -(f + f_p)(v, u) \end{aligned}$$



New valid flow

引理

$$f' = f + f_p: V \times V \rightarrow \mathbb{R} \quad \text{in } G$$

流守恒性质:

任给 $u \in V - \{s, t\}$, 有 $\sum_{v \in V} f'(u, v) = 0$

Proof:

$$\begin{aligned} u \in V - \{s, t\} &\Rightarrow \sum_{v \in V} (f + f_p)(u, v) = \sum_{v \in V} (f(u, v) + f_p(u, v)) \\ &= \sum_{v \in V} f(u, v) + \sum_{v \in V} f_p(u, v) = 0 + 0 = 0 \end{aligned}$$



New valid flow

引理

$$|f + f_p| = |f| + |f_p|$$

Proof:

$$\begin{aligned} |f + f_p| &= \sum (f + f_p)(s, v) = \sum (f(s, v) + f_p(s, v)) \\ &= \sum f(s, v) + \sum f_p(s, v) = |f| + |f_p| \end{aligned}$$



New valid flow

推论9.1

$$f' = f + f_p: V \times V \rightarrow \mathbf{R} \quad \text{in } G$$

$$|f + f_p| = |f| + |f_p| > |f|$$

- **推论9.1表明：**如果能找到一条增广路径，我们就可以增加 f_p 到 f 上，从而可以得到流量值更大的网络 G 。重复这个过程，我们就能不断地增加网络的流量。
- **Question：**如果在剩余网络中再也找不到增广路径，那么所得流就是最大流吗？ 的流最大是否等价于剩余网络不再包含增广路径？

➤➤➤ 流网络的割 (Cuts of flow networks)

- **定义9.7:** 给定一个网络 $G=(V,E)$, 网络的一个割 (S,T) 是把网络 G 的顶点集 V 分成两个子集 S 和 $T=V-S$ 且 $s \in S, t \in T$, 其中割 (S,T) 的容量记为 $c(S,T)$, 定义为

$$c(S,T) = \sum_{u \in S, v \in T} c(u,v)$$

流过割 (S,T) 的流量记为 $f(S,T)$, 定义为

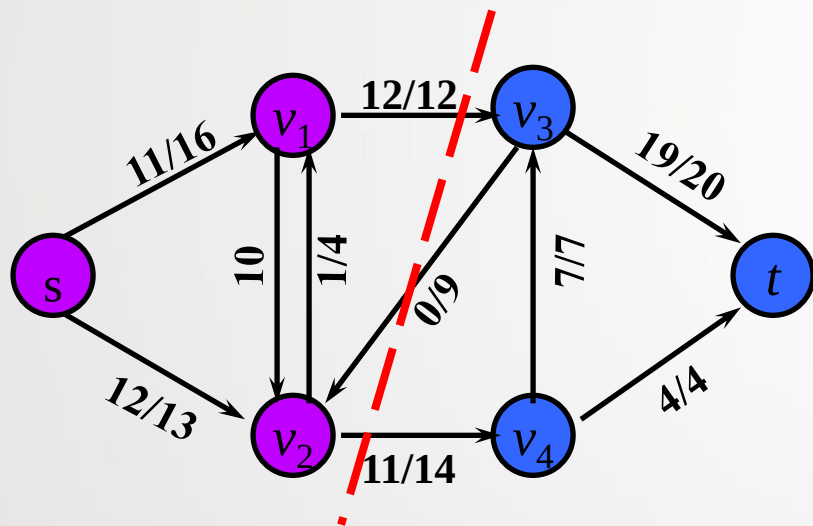
$$f(S,T) = \sum_{u \in S, v \in T} f(u,v)$$



流网络的割 (Cuts of flow networks)

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

$$f(S, T) = \sum_{u \in S, v \in T} f(u, v)$$



$$S = \{s, v_1, v_2\}, T = \{v_3, v_4, t\}$$

$$c(S, T) = c(v_1, v_3) + c(v_2, v_4) = 12 + 14 = 26$$

$$f(S, T) = f(v_1, v_3) + f(v_2, v_4) + f(v_2, v_3) = 12 + 11 + (-0) = 23$$

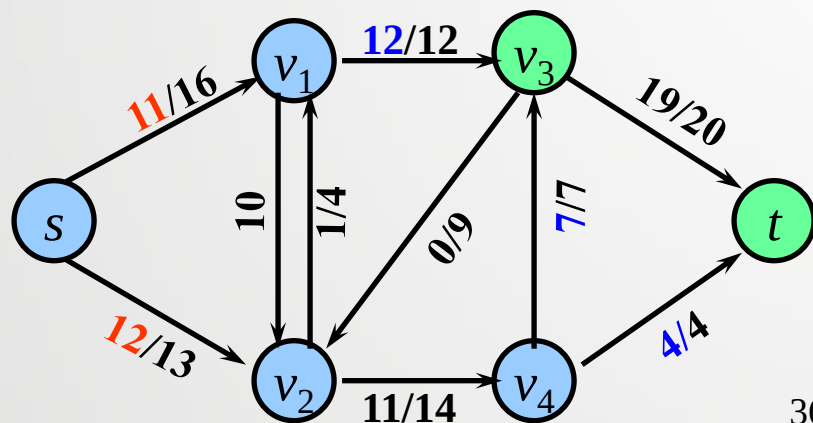
流网络的割 (Cuts of flow networks)

- **引理9.3** 给定一个网络 G ，令 f 为 G 的流。对于 G 的任意一个割 (S, T) ，有 $f(S, T) = |f|$ 。

证明：对 S 的顶点数进行归纳证明。

如果 $S=\{s\}$ ，由定义9.2显然成立。

假定对割 (S, T) 成立，则可以证明当 $w \in T - \{t\}$ ，结论对割 $(S \cup \{w\}, T - \{w\})$ 也成立。事实上，



36

$$\begin{aligned} f(S \cup \{w\}, T - \{w\}) &= f(S, T - \{w\}) + f(w, T - \{w\}) \\ &= f(S, T) - f(S, w) + f(w, T) - f(w, w) \\ &= f(S, T) - f(S, w) + f(w, T) - 0 \\ &= f(S, T) + f(w, S) + f(w, T) \\ &= f(S, T) + f(w, V) \\ &= f(S, T) = |f| \end{aligned}$$

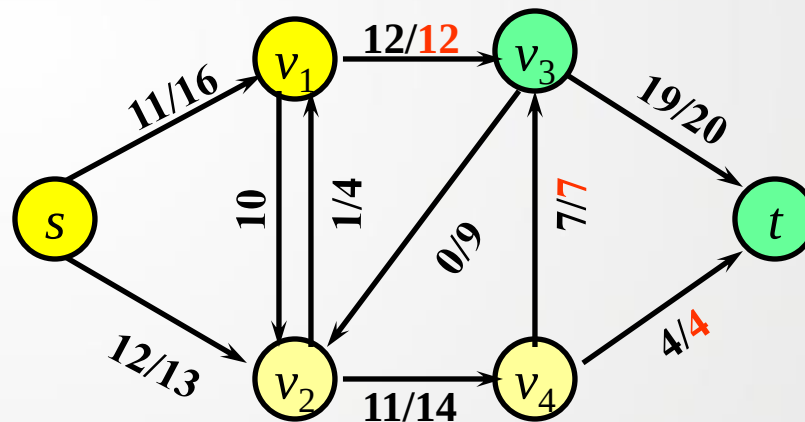


流网络的割 (Cuts of flow networks)

- **推论9.2** 网络 G 中任意流 f 的值被 G 的任意割的容量所限制，即 $|f| \leq c(S, T)$

Proof. $|f| = f(S, T)$

$$\begin{aligned} &= \sum_{u \in S} \sum_{v \in T} f(u, v) \\ &\leq \sum_{u \in S} \sum_{v \in T} c(u, v) \\ &= c(S, T) \end{aligned}$$





最大流最小割定理

- **定理9.1** 给定一个网络 $G = (V, E)$, 令 f 为 G 的流, 则下列三个结论相互等价:
 - (1) 存在一个割 (S, T) , 使得 $|f| = c(S, T)$ 。
 - (2) f 是 G 的最大流;
 - (3) 剩余网络 G_f 不存在增广路径;

证明:

(1) $\Rightarrow$ (2): 由于对任意割 (S, T) , 有 $|f| = f(S, T) \leq c(S, T)$, $|f| = c(S, T)$ 意味着 f 是最大流



最大流最小割定理

- **定理9.1** 给定一个网络 $G = (V, E)$, 令 f 为 G 的流, 则下列三个结论相互等价:
 - (1) 存在一个割 (S, T) , 使得 $|f| = c(S, T)$ 。
 - (2) f 是 G 的最大流;
 - (3) 剩余网络 G_f 不存在增广路径;

proof: (2) $\Rightarrow$ (3):

利用反证法。已知 f 是 G 的最大流, 假设剩余网络 G_f 存在增广路径 p , 则可以通过公式 $f' = f + f_p$, 增加 G 的流, 按照推论9.1, 这样就可以得到一个比 f 更大的流, 与假设 f 是最大流相矛盾, 故所证成立。



最大流最小割定理

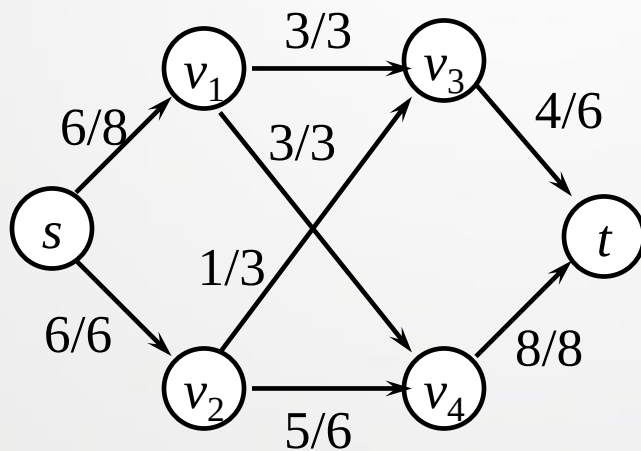
- **定理9.1** 给定一个网络 $G = (V, E)$, 令 f 为 G 的流, 则下列三个结论相互等价:

(1) 存在一个割 (S, T) , 使得 $|f| = c(S, T)$ 。

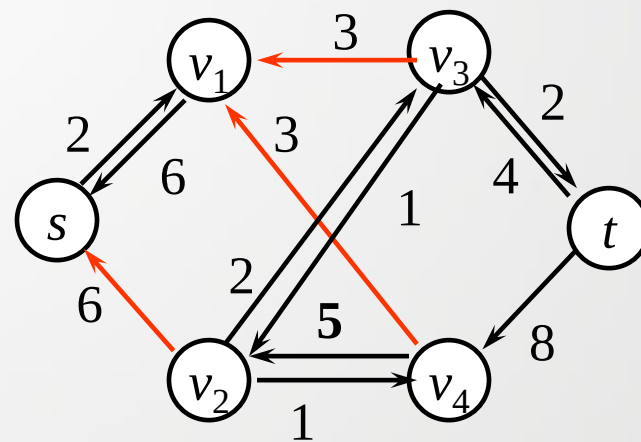
(2) f 是 G 的最大流;

(3) 剩余网络 G_f 不存在增广路径;

proof: (3) $\Rightarrow$ (1): 定义 $S = \{v \in V: G_f \text{ 中存在从 } s \text{ 到 } v \text{ 的路径}\}$, $T = V - S$



original flow network G



residual network G_f



最大流最小割定理

- **定理9.1** 给定一个网络 $G = (V, E)$, 令 f 为 G 的流, 则下列三个结论相互等价:

(1) 存在一个割 (S, T) , 使得 $|f| = c(S, T)$ 。

(2) f 是 G 的最大流;

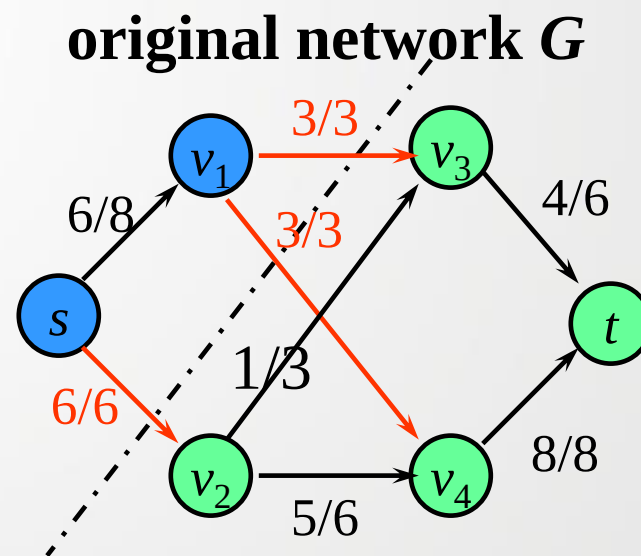
(3) 剩余网络 G_f 不存在增广路径;

续前:

所有的从 S 到 T 边均饱和,

即对 $\forall u \in S, v \in T: f(u, v) = c(u, v)$
(otherwise $(u, v) \in E_f$ and $v \in S$)

$\Rightarrow |f| = f(S, T) = c(S, T)$



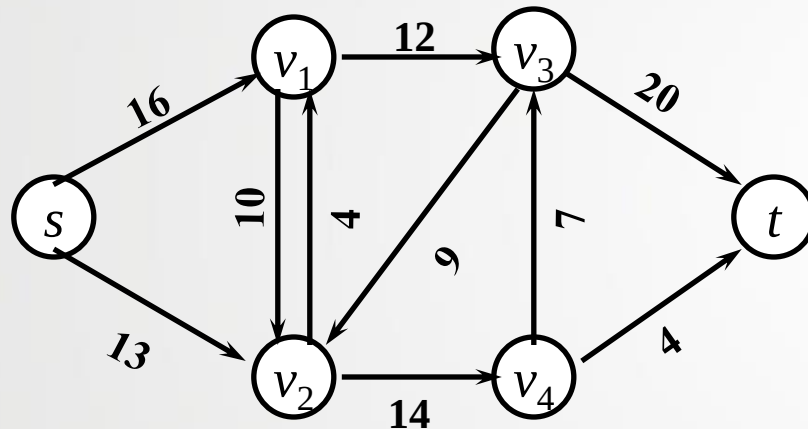
FordFulkerson方法

FordFulkerson(G, s, t, c)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f

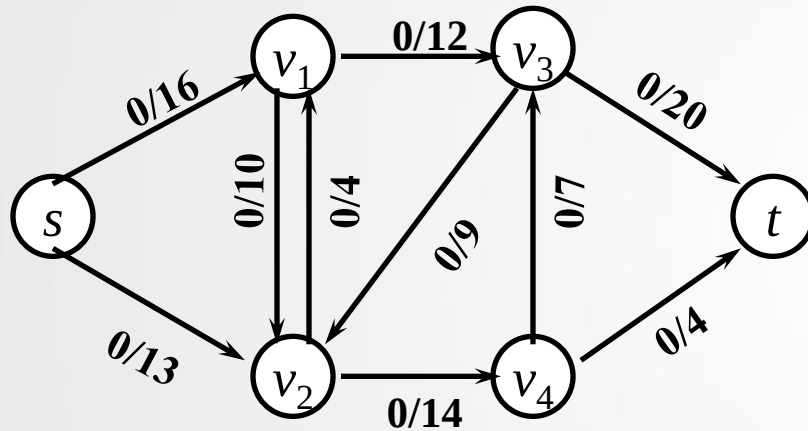


FordFulkerson(G, s, t, c)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f

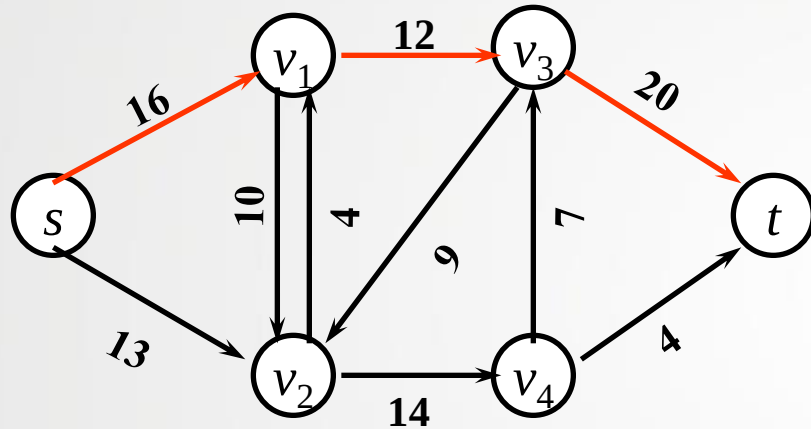


FordFulkerson(G, s, t, c)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f

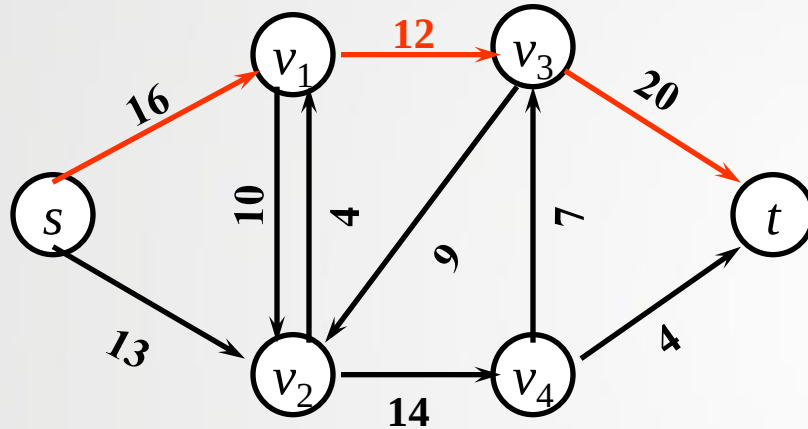


FordFulkerson(G, s, t, c)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f



FordFulkerson(G, s, t, c)

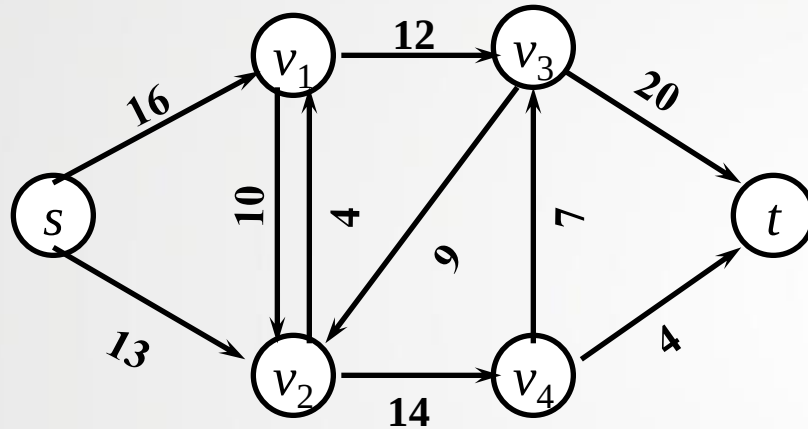
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

bottleneck capacity:

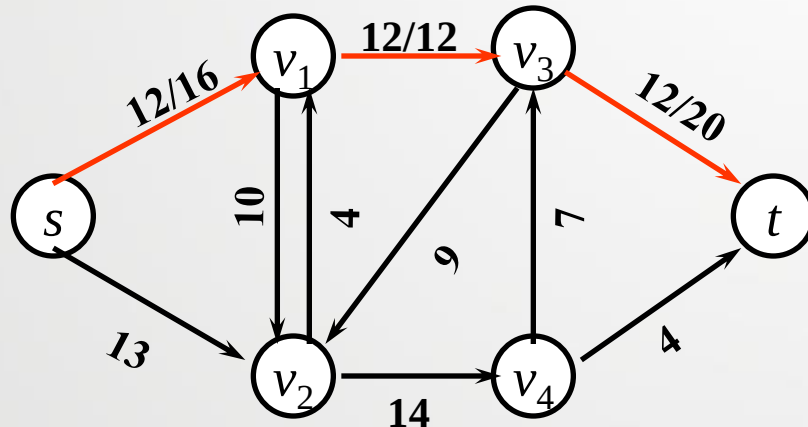
$$c_f(p) = 12$$

Example of an execution

Residual network G_f



New flow network G



FordFulkerson(G, s, t, c)

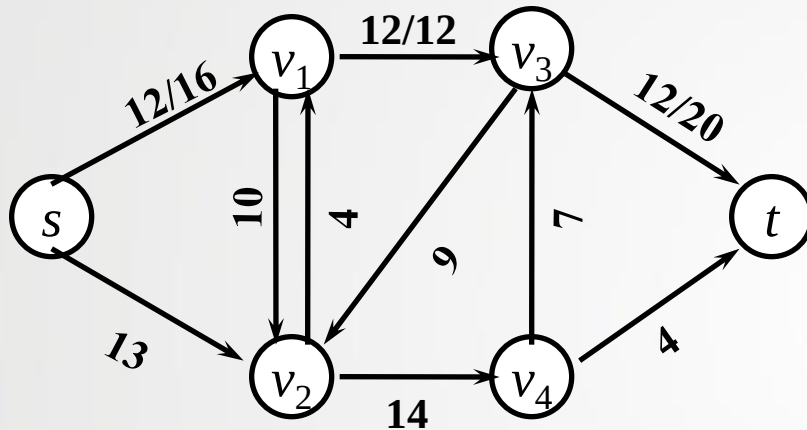
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

bottleneck capacity:

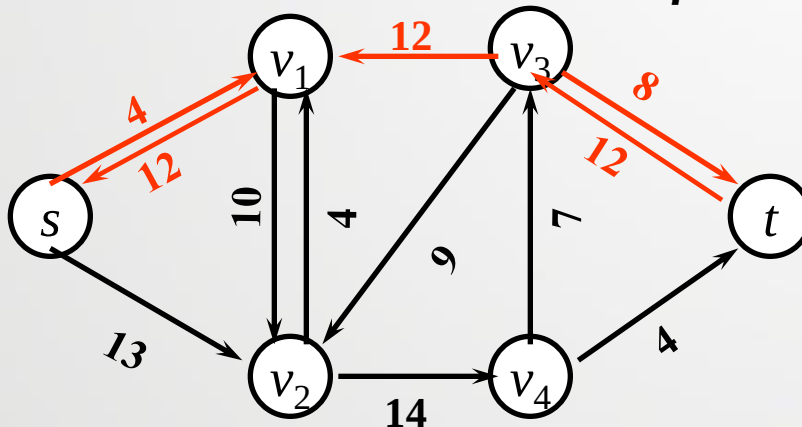
$$c_f(p) = 12$$

Example of an execution

New flow network G



Residual network G_f



FordFulkerson(G, s, t, c)

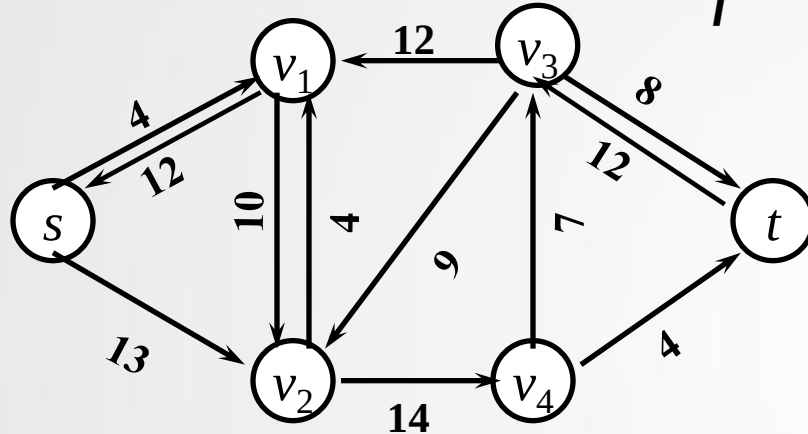
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

bottleneck capacity:

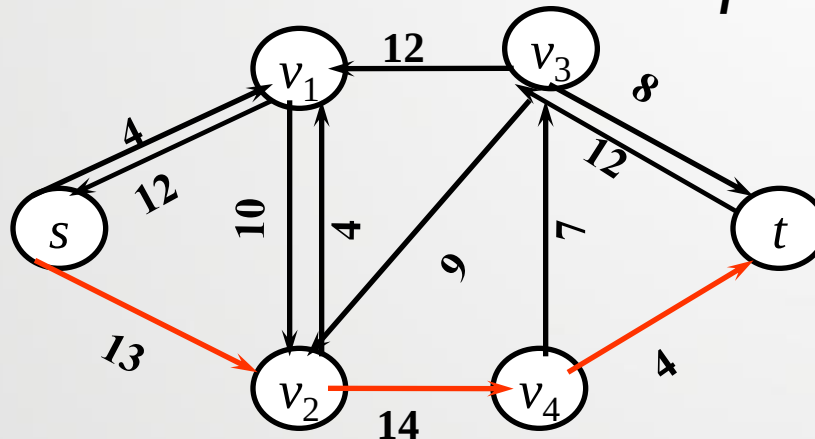
$$c_f(p) = 12$$

Example of an execution

Residual network G_f



Residual network G_f

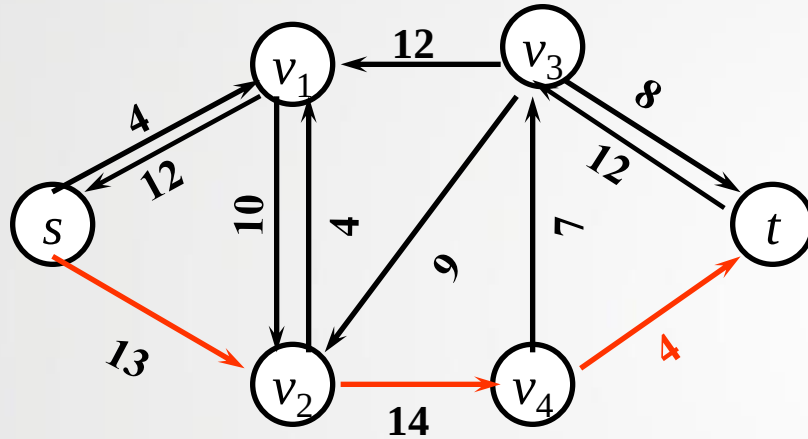


FordFulkerson(G, s, t, c)

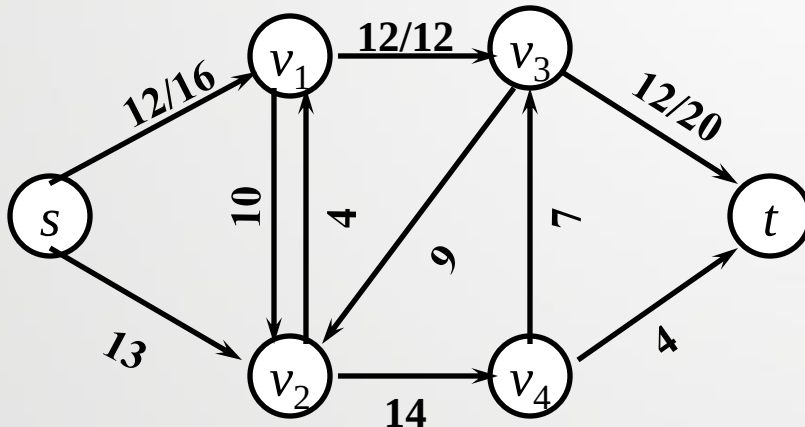
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f



New flow network G



FordFulkerson(G, s, t, c)

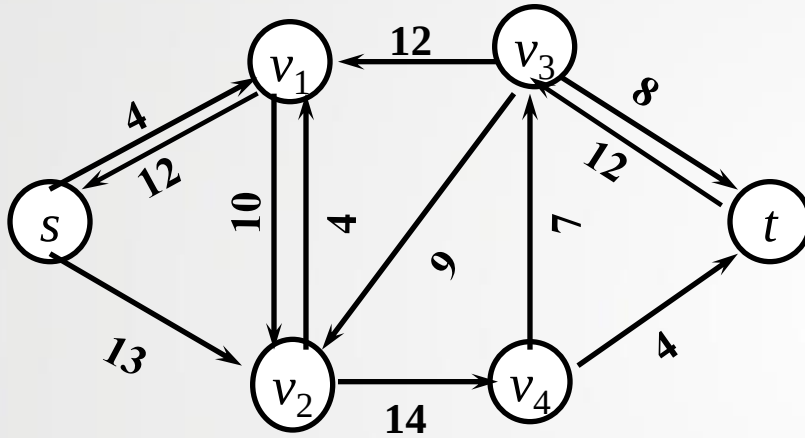
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

bottleneck capacity:

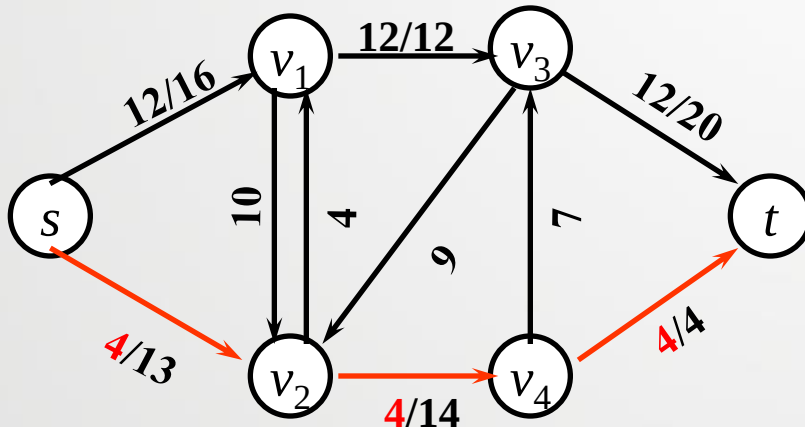
$$c_f(p) = 4$$

Example of an execution

Residual network G_f



New flow network G



FordFulkerson(G, s, t, c)

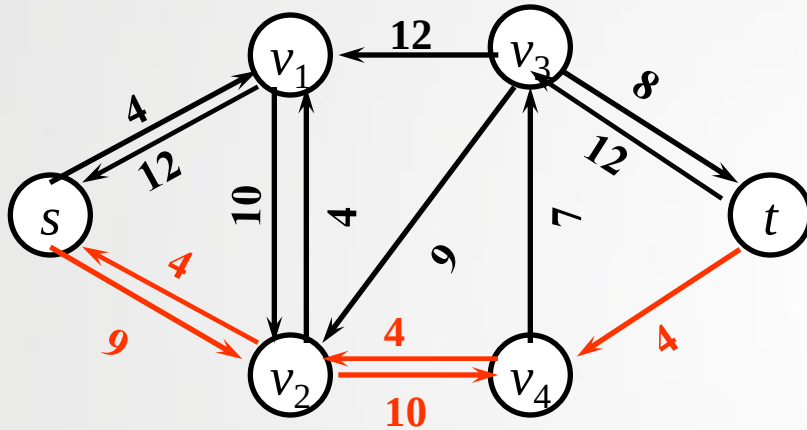
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

bottleneck capacity:

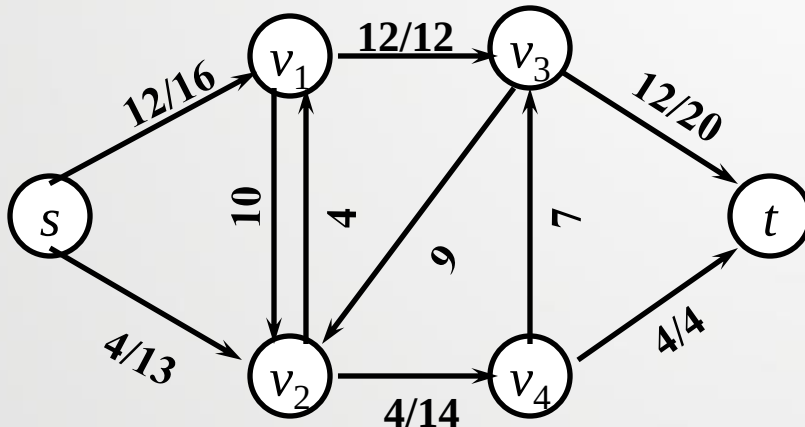
$$c_f(p) = 4$$

Example of an execution

Residual network G_f



New flow network G

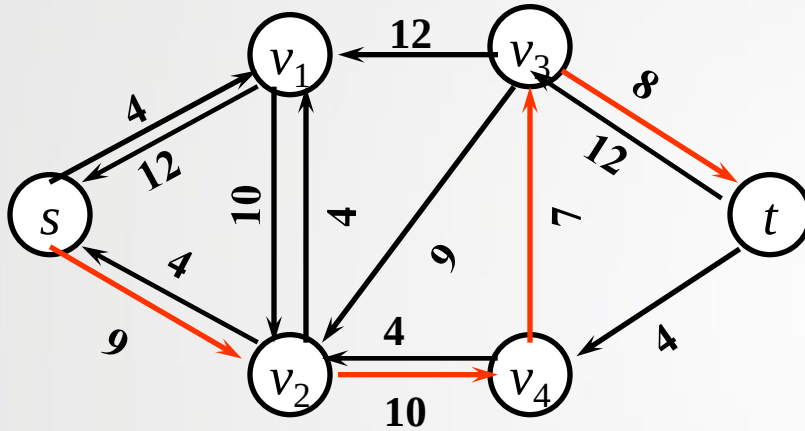


FordFulkerson(G, s, t, c)

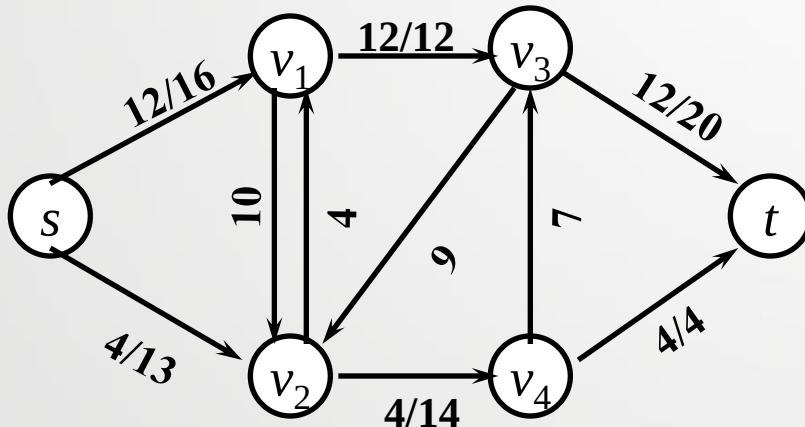
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f



New flow network G

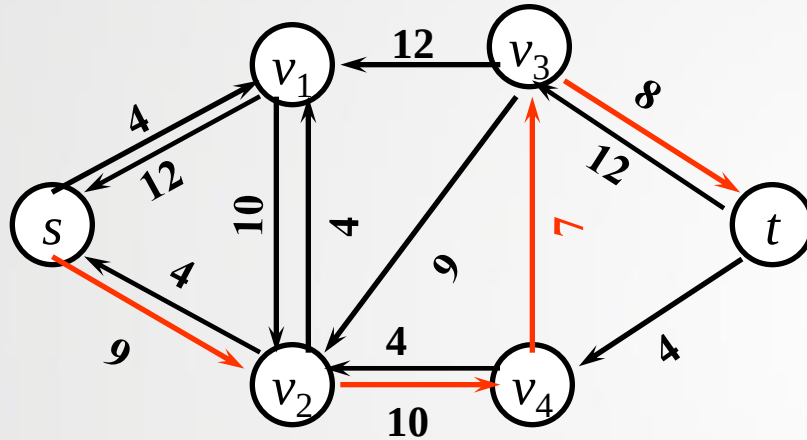


FordFulkerson(G, s, t, c)

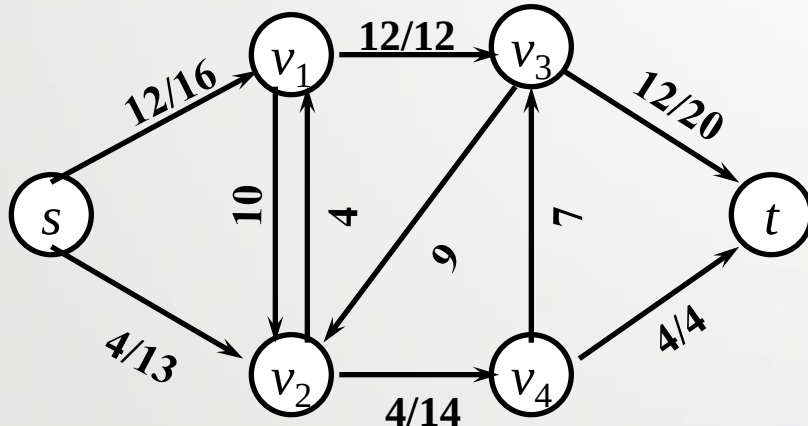
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f



New flow network G



FordFulkerson(G, s, t, c)

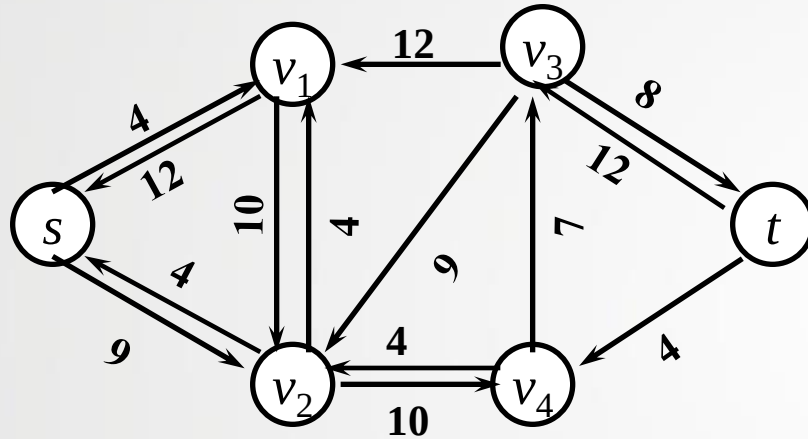
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

bottleneck capacity:

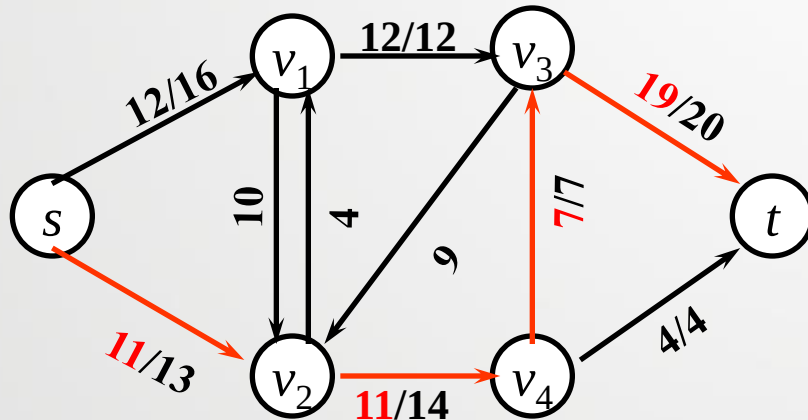
$$c_f(p) = 7$$

Example of an execution

Residual network G_f



New flow network G



FordFulkerson(G, s, t, c)

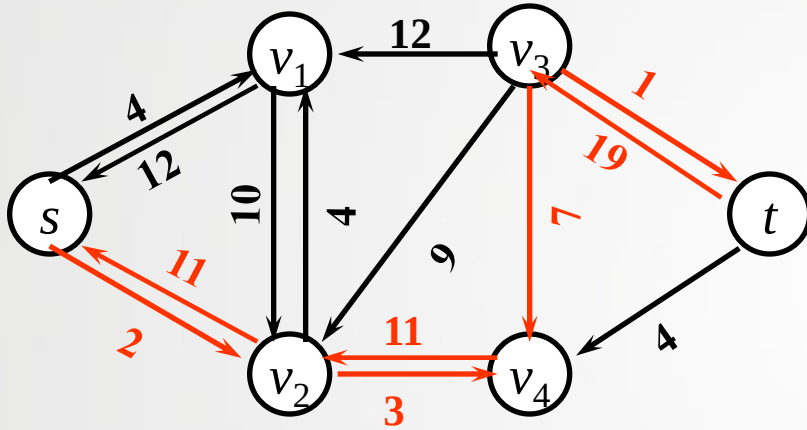
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

bottleneck capacity:

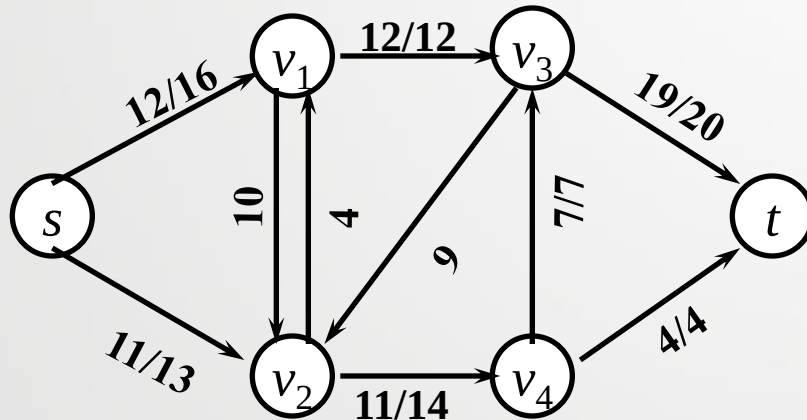
$$c_f(p) = 7$$

Example of an execution

Residual network G_f



New flow network G

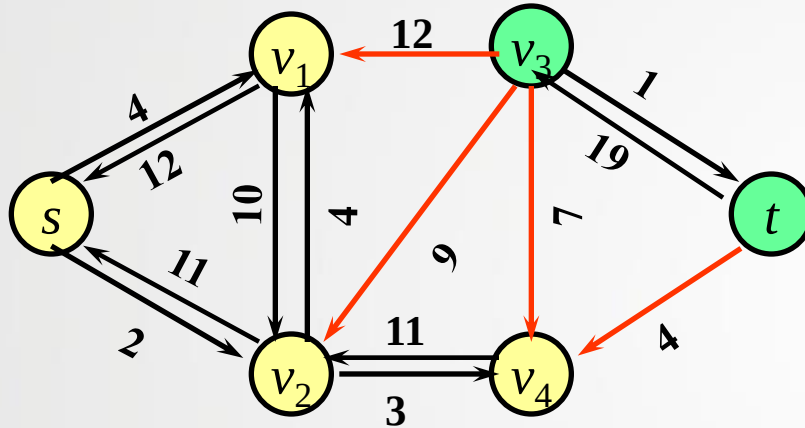


FordFulkerson(G, s, t, c)

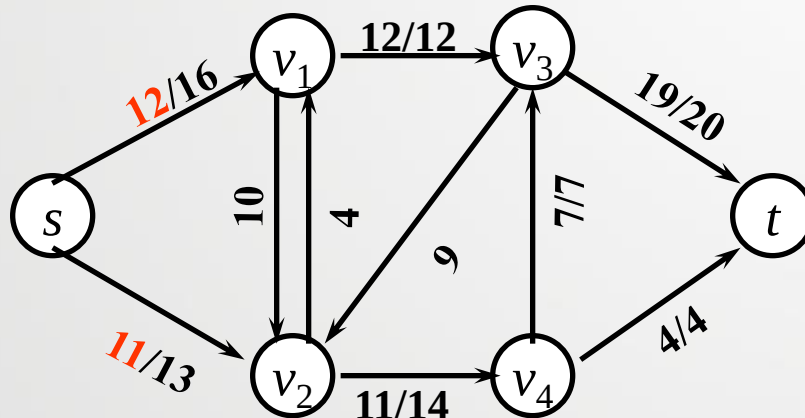
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Example of an execution

Residual network G_f



New flow network G



FordFulkerson(G, s, t, c)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

Finally we have:

$$|f| = f(s, V) = 23$$



FordFulkerson方法的分析

FordFulkerson(G, s, t, c)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 **while** there is an augmenting path p in G_f **do**
- 5 Let $c_f(p)$ be the bottleneck capacity of p
- 6 **for** each edge (u,v) in p **do**
- 7 $f(u,v) \leftarrow f(u,v) + c_f(p)$
- 8 Update the residual graph G_f

运行时间取决于行4中，增广路径 p 该如何确定



FordFulkerson方法的分析

FordFulkerson(G, s, t, c)

1 **for** each edge $(u,v) \in E$ **do**

2 $f(u,v) \leftarrow 0$

3 $G_f \leftarrow G$

4 **while** there is an augmenting path p in G_f **do**

5 Let $c_f(p)$ be the bottleneck capacity of p

6 **for** each edge (u,v) in p **do**

7 $f(u,v) \leftarrow f(u,v) + c_f(p)$

8 Update the residual graph G_f

} $O(|E|)$

$O(|E|)$
 $O(|E|)$
[$O(|E|)$] } $O(|E| |f_{max}|)$
 $O(|E|)$

running time: $O(|E| |f_{max}|)$ with f_{max} as maximum flow

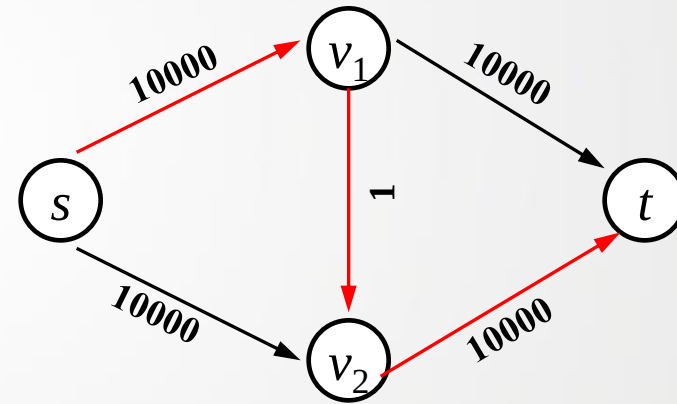
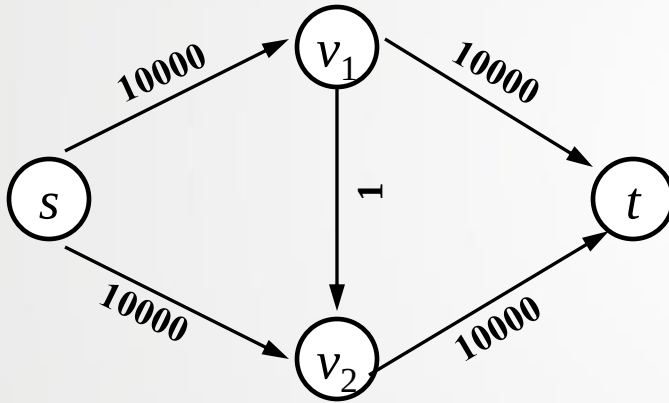
注意：增广路径被任意的选择，容量是整数

Analysis of the Ford Fulkerson algorithm

Running time (arbitrary choice of p)

任意选择的结果:

如果 $|f^*|$ 比较大:



running time: $O(|E| |f_{\max}|)$ with $f_{\max}$ as maximum flow

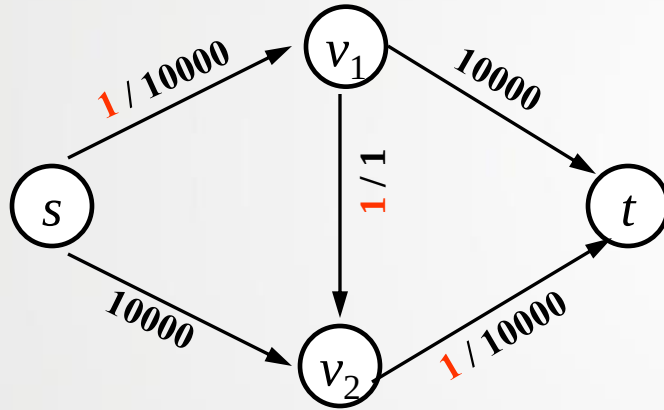
注意: 增广路径被任意的选择, 容量是整数

Analysis of the Ford Fulkerson algorithm

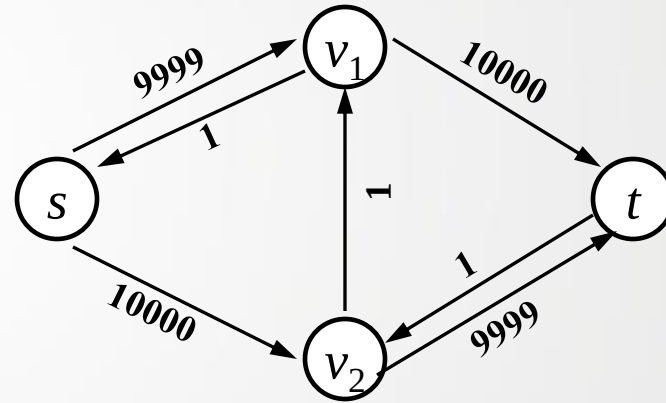
Running time (arbitrary choice of p)

任意选择的结果:

如果 $|f^*|$ 比较大:



residual network G_f



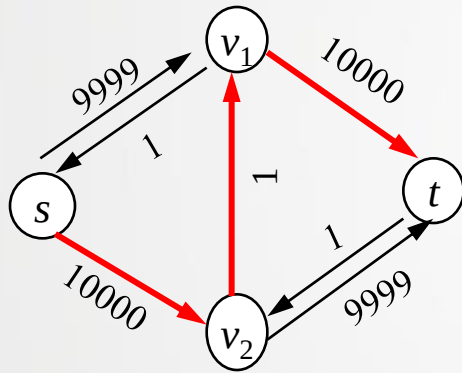
running time: $O(|E| |f_{\max}|)$ with
 $f_{\max}$ as maximum flow

Analysis of the Ford Fulkerson algorithm

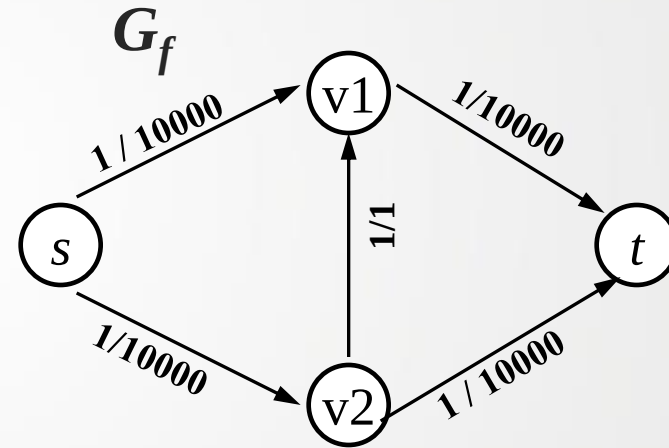
Running time (arbitrary choice of p)

任意选择的结果:

如果 $|f^*|$ 比较大:



residual network



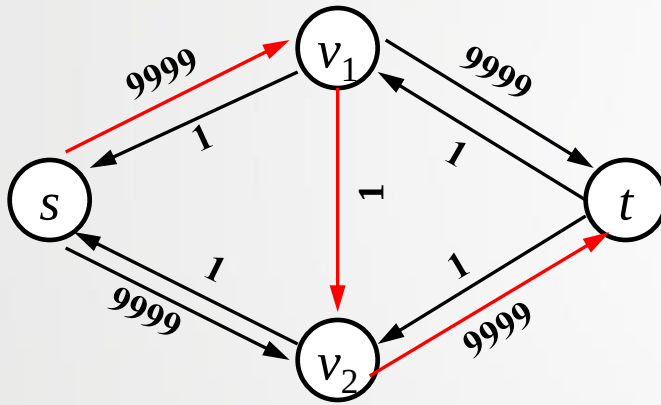
running time: $O(|E| |f_{\max}|)$ with $f_{\max}$ as maximum flow

Analysis of the Ford Fulkerson algorithm

Running time (arbitrary choice of p)

任意选择的结果:

如果 $|f^*|$ 比较大:



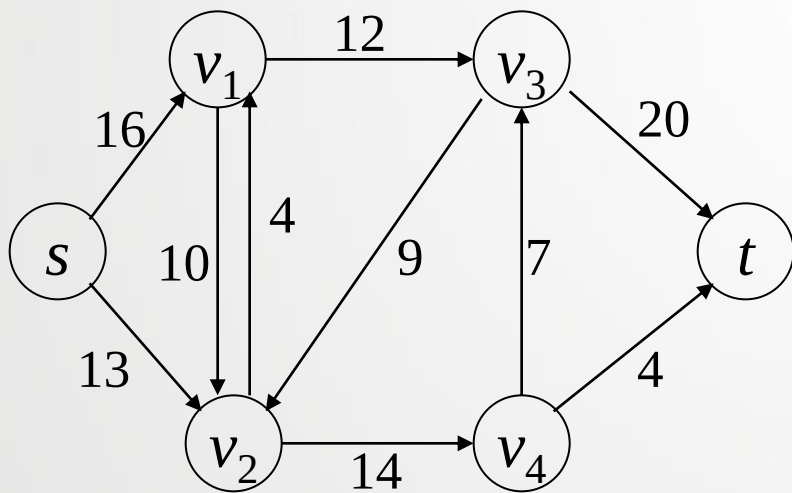
运行时间: 执行 20000次

running time: $O(|E| |f_{\max}|)$ with $f_{\max}$ as maximum flow

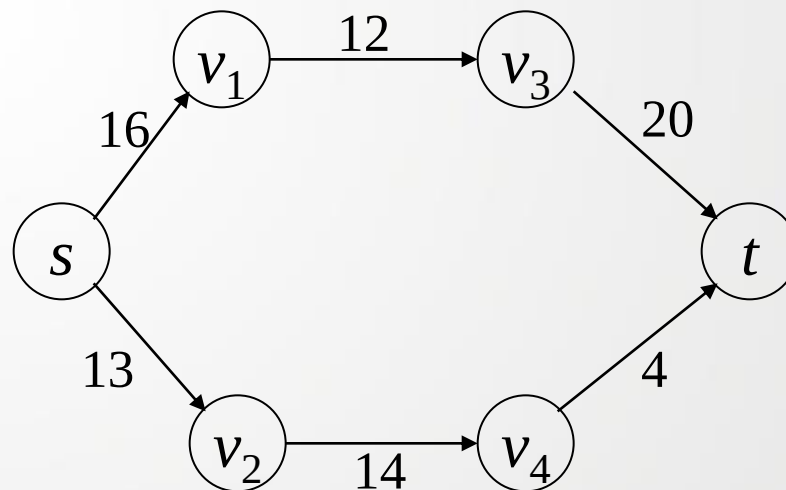


最短路径增广法 (SPA)

- **定义9.8** 给定一个网络 $G=(V,E)$, 定义层次图(level graph) $G_L=(V, E')$, 其中 $E'=\{(u,v): (u,v) \in E, \text{ 且 } d(v)=d(u)+1\}$, $d(v)$ 也称为顶点 v 的层次, 是从源点 s 到 v 的路径中最短路径的边数。



$G=(V,E)$



层次图 G_L



最短路径增广法 (SPA)

- 利用宽度优先算法来找到层次图，找到了层次图，我们就知道了找增广路径的方法。即我们利用宽度优先搜索找到具有最短路径长度的增广路径，并在当前的流上，沿着增广路径增广个单位的流量。



最短路径增广法 (SPA)

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



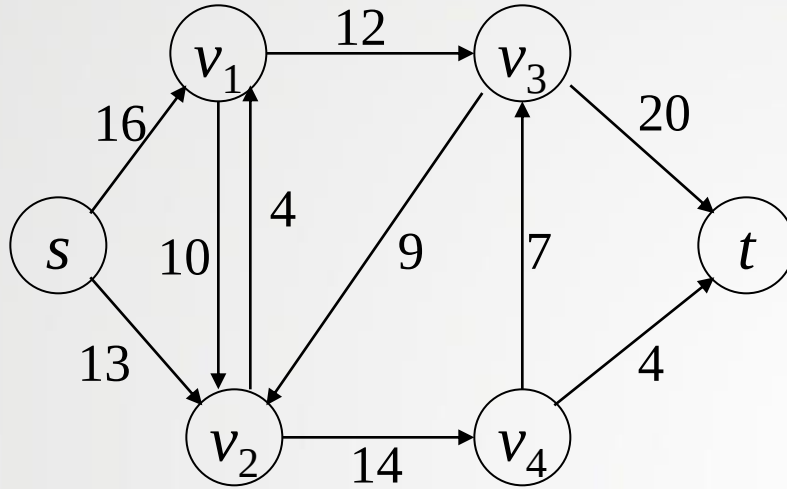
最短路径增广法 (SPA)

ShortestPathAugmentation算法开始先初始化流为零流，并设网络的剩余网络 G_f 为原始图，并计算其层次图（level graph），然后执行while循环。while循环分阶段进行，每个阶段由下面两步组成：

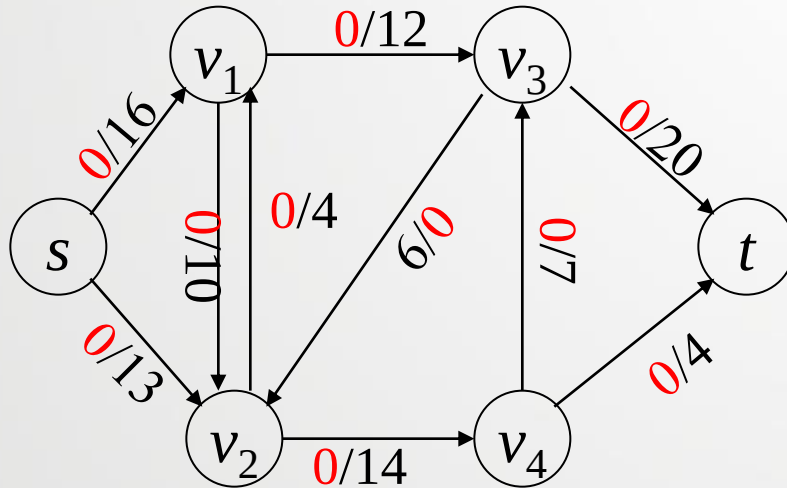
- ① 只要 G_L 中有从 s 到 t 的路径，就用 f_p 对当前的流 f 进行增广，即 $f+f_p$ ，从 G_L 和 G_f 中移去饱和边，并相应地更新 G_L 和 G_f ；
- ② 根据剩余网络 G_f 计算出层次图，若 t 不在 G_L 中，则停止，否则继续。



Example of an execution



Input network $G=(V,E)$



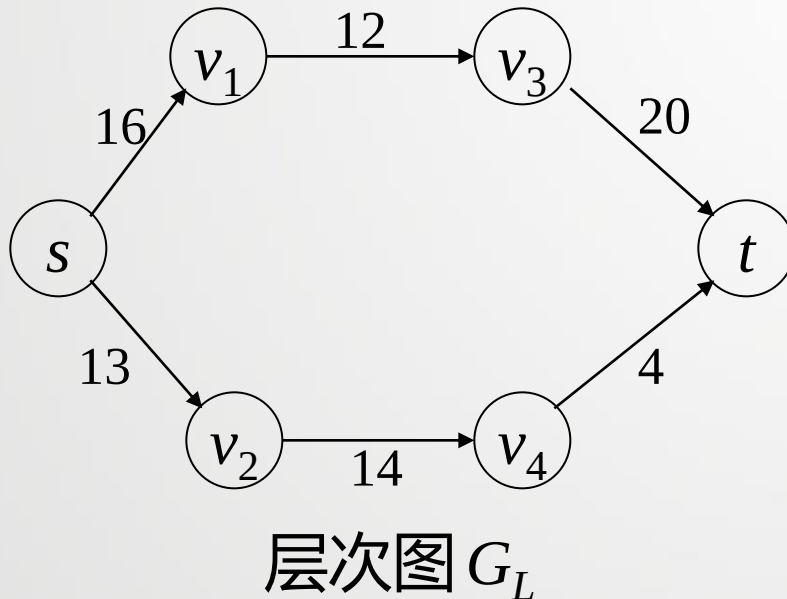
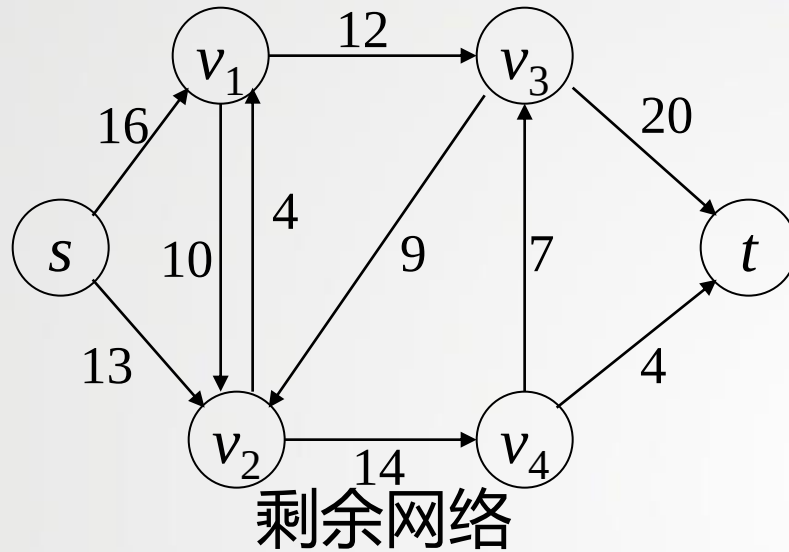
Input network $G=(V,E)$

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution

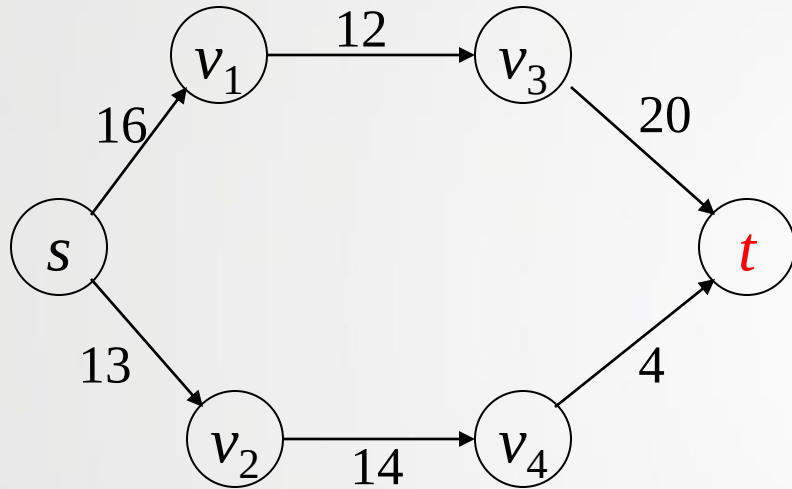


ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



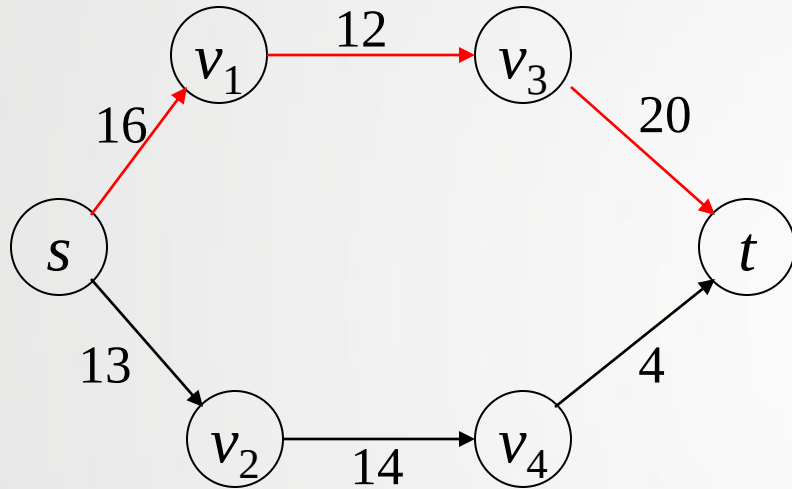
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



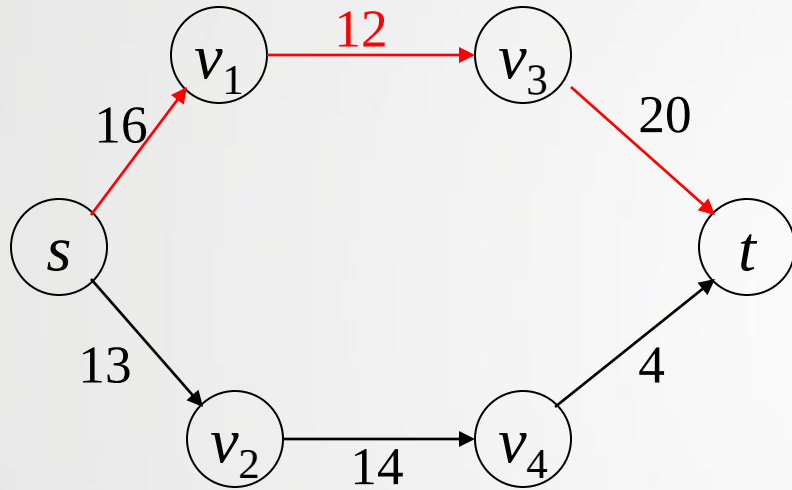
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



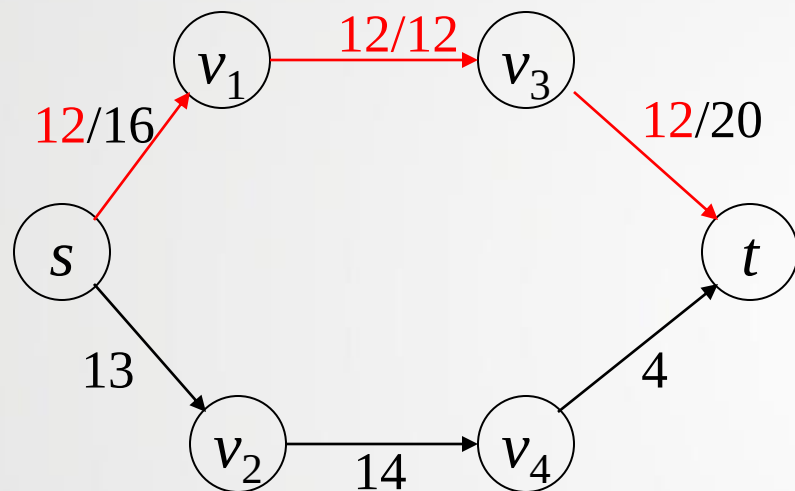
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



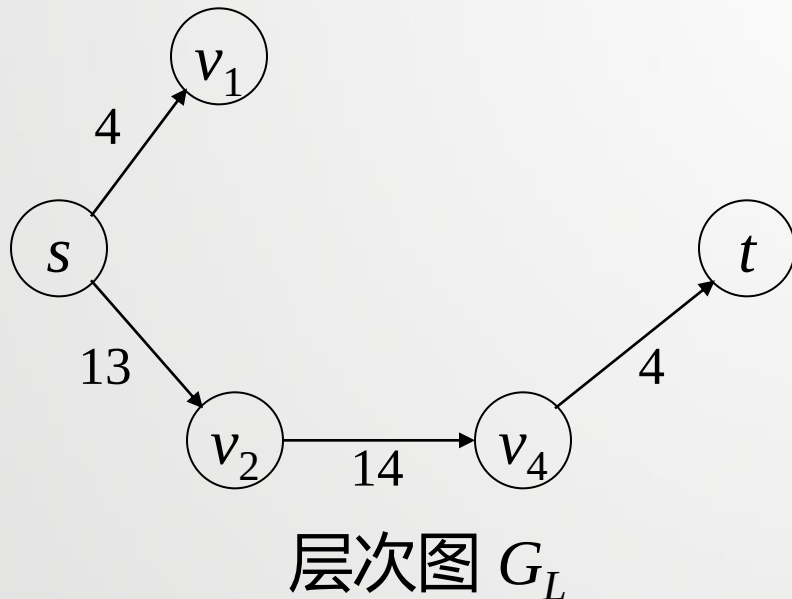
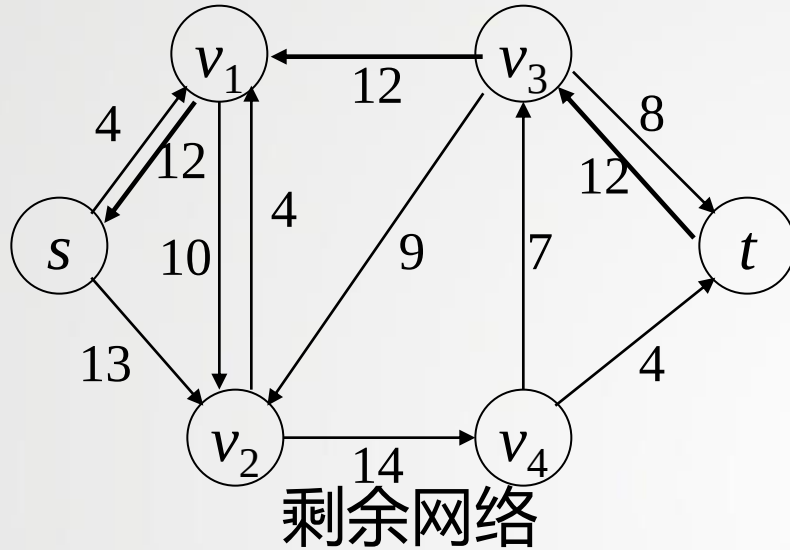
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution

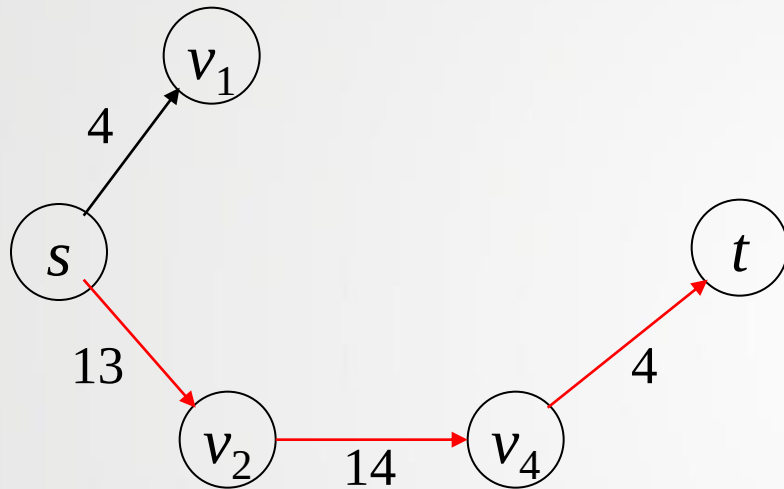


ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



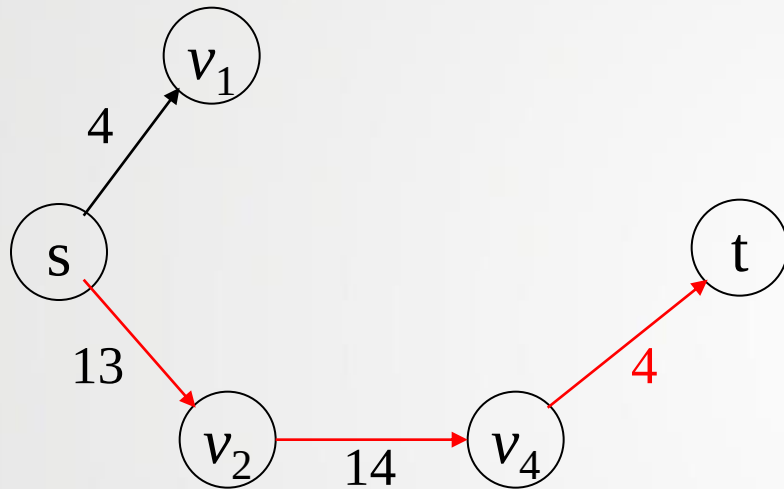
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



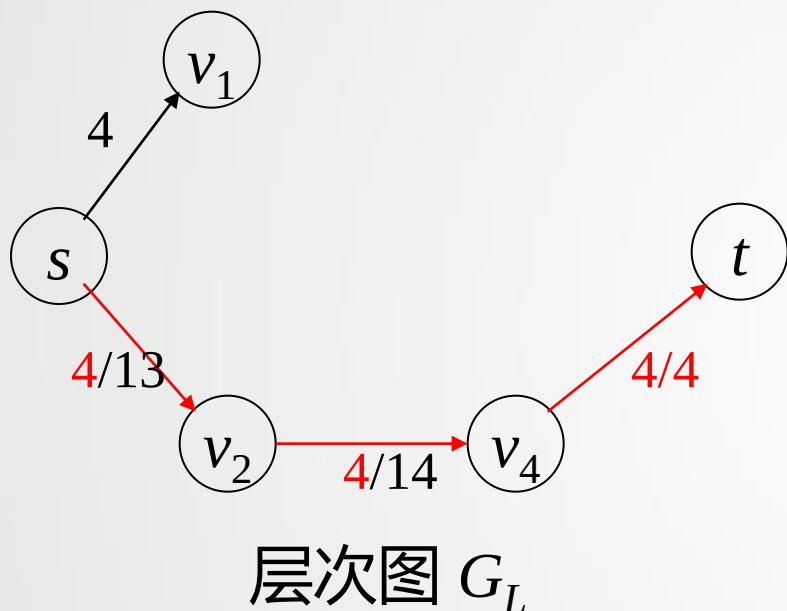
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution

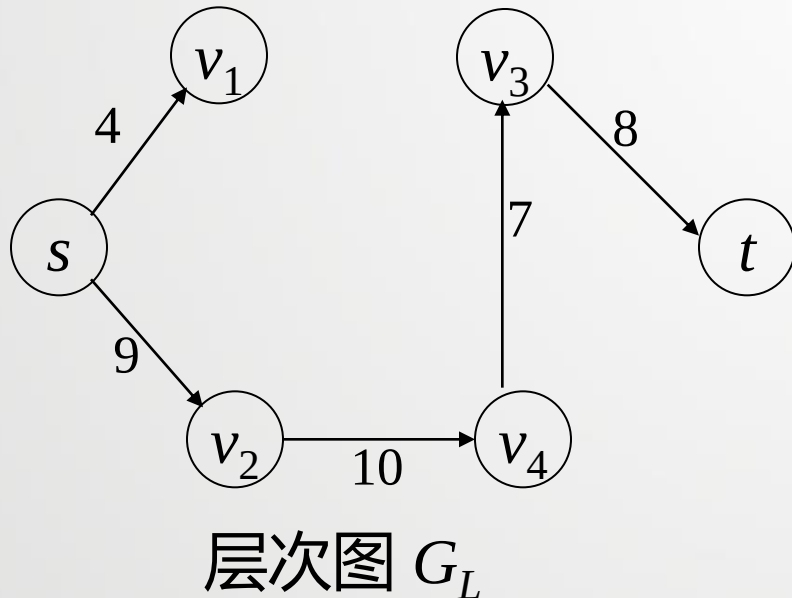
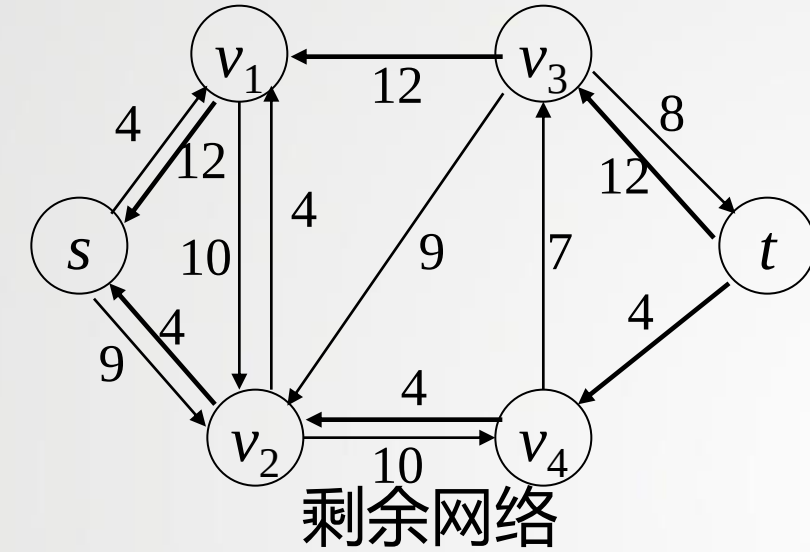


ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution

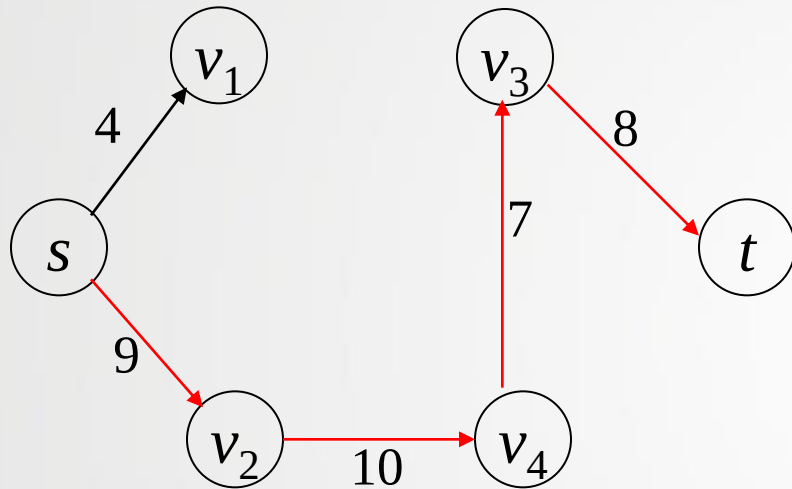


ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



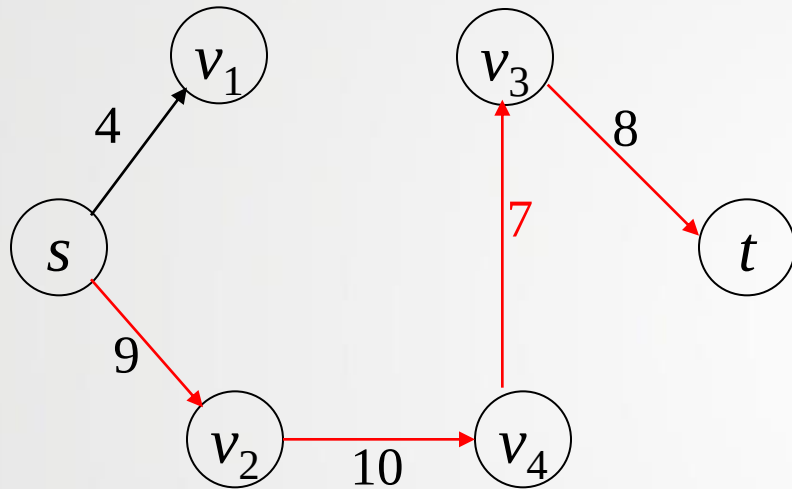
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution

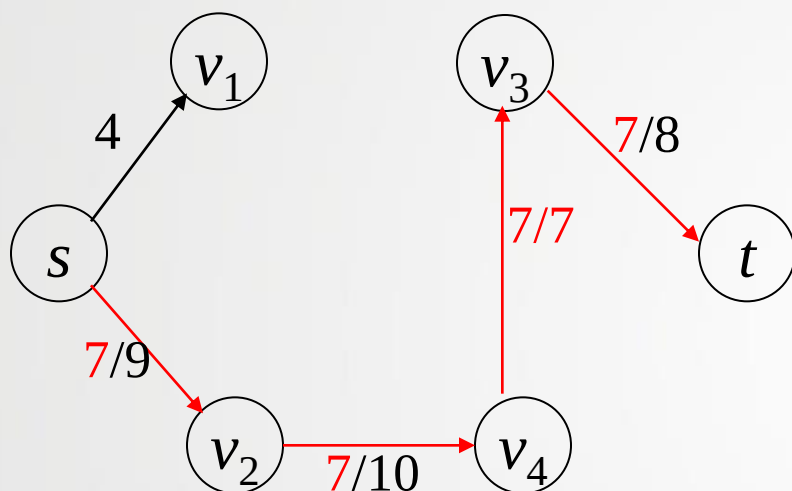


层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L

Example of an execution



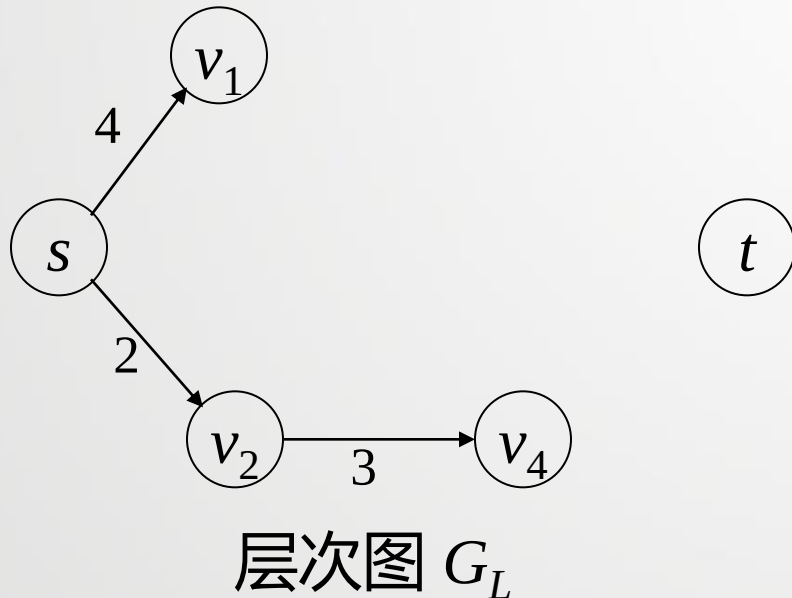
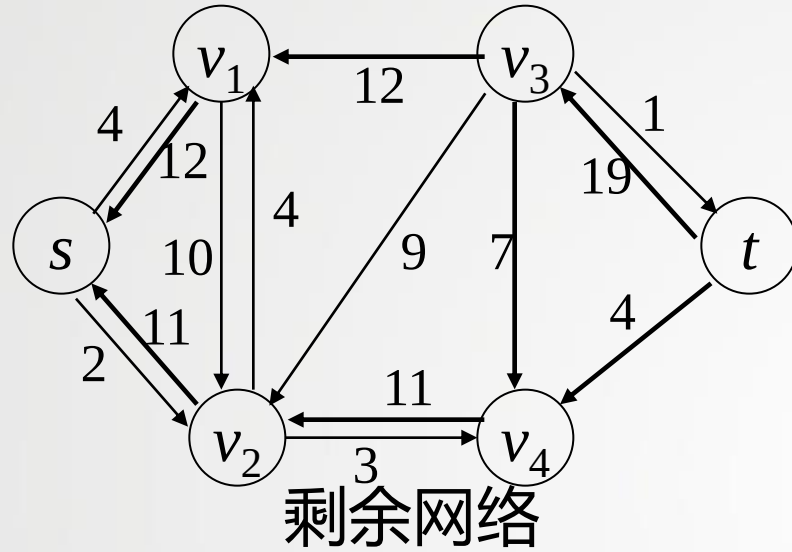
层次图 G_L

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution

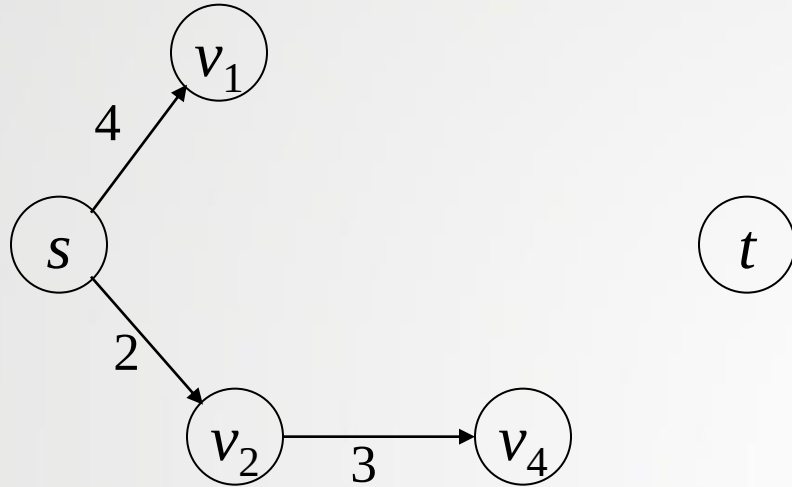


ShortestPathAugmentation(G, s, t)

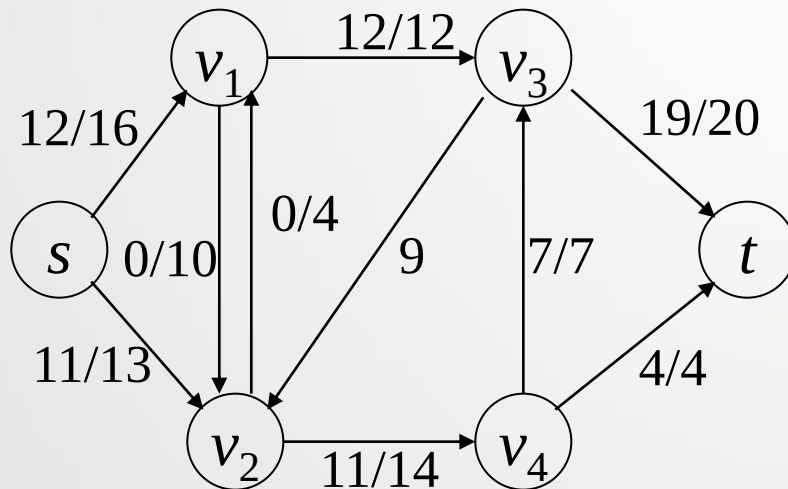
- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L



Example of an execution



层次图 G_L



New flow network G

ShortestPathAugmentation(G, s, t)

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 **while** there is a path p from s to t in G_L **do**
- 7 let $c_f(p)$ be the bottleneck capacity on p
- 8 augment the current flow f by $c_f(p)$
- 9 update G_L and G_f along the path p
- 10 use G_f to compute a new level graph G_L

最后求得: $|f| = f(s, V) = 23$



最短路径增广算法分析

引理9.4 在ShortestPathAugmentation(G, s, t) 算法中, 第5行的while循环最多执行次。

证明: 我们只需要证明ShortestPathAugmentation算法产生的层次图至多为 $|V|$ 个即可。

首先, 我们证明该算法在前后层次图中找到的增广路径的长度序列是严格递增的。设 p 为当前层次图中任意一条增广路径, 在对 p 增加流量后, 路径 p 中至少有一条边将饱和, 并在剩余网络中消失。至多 $|p|$ 条新边会在剩余网络中出现, 但它们是后向边, 因此不会影响从 s 到 t 的最短路径。当沿着同一层次图中所有增广路径对流依次进行增广后, 下一层次图的增广路径会更长, 故所证成立。



最短路径增广算法分析

- 当 t 在 G_L 中再也不能从 s 到达时，则找不到增广路径。因为任何增广路径长度在1和 $|V|-1$ 间，因此用做增广的层次图个数最多为 $|V|-1$ 。因为 t 不出现的层次图也要计算一次，因此计算的层次图总数最多是 $|V|$ 。



最短路径增广算法分析

定理9.2

- 给定一个网络 $G=(V,E)$ ，用ShortestPathAugmentation(G, s, t) 算法找到最大流所需要的时间为 $O(|V||E|^2)$ 。

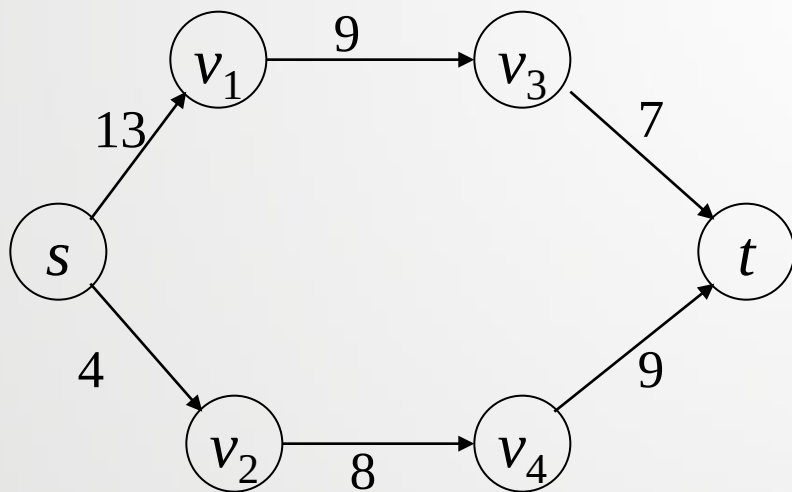
证明：

- 因为每次有一条边在层次图中消失，因此层次图中最多可能有 $|E|$ 条增广路径，即沿着同样长度的路径至多只能增广 $|E|$ 次，根据引理9.4，用来增广而计算的层次图个数最多是 $|V|$ ，因此，增广的次数最多为 $|V||E|$ 。使用宽度优先搜索在 G_L 中找到一条最短增广路径需要的时间 $O(|E|)$ 。那么，寻找所有增广路径的时间为 $O(|V||E|^2)$ ，因此用ShortestPathAugmentation(G, s, t) 算法找到最大流所需要的时间为 $O(|V||E|^2)$ 。

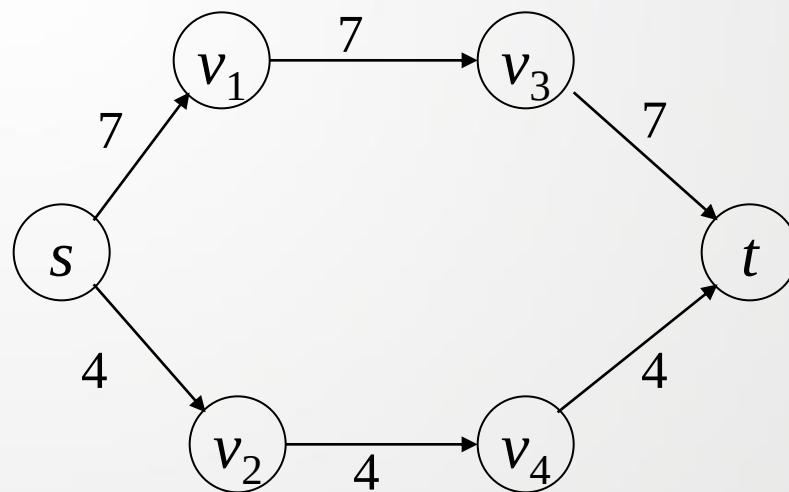


Dinic 算法

- **定义9.9** 给定一个网络 $G=(V,E)$, G_L 是 G 的子图, 且包含 s 和 t , f 是 G_L 的流, 如果在 G_L 中每一条从 s 到 t 的路径中都至少有一条饱和边, 则称 G_L 的流 f 为阻塞流。
- **Dinic算法**就是找出层次图, 然后按照深度优先的方式找增广路径, 并按照阻塞流对当前流进行增广。



层次图 G_L



阻塞流



Dinic 算法

Dinic()

- 1 **for** each edge $(u,v) \in E$ **do**
- 2 $f(u,v) \leftarrow 0$
- 3 $G_f \leftarrow G$
- 4 find the level graph G_L of G_f
- 5 **while** t is a vertex in G_L **do**
- 6 $u \leftarrow s; p \leftarrow \{u\}$
- 7 **while** $\text{outdegree}(s) > 0$ **do**
- 8 **while** $u \neq t$ and $\text{outdegree}(s) > 0$ **do**
- 9 **if** $(\text{outdegree}(u) > 0)$ **then**
- 10 let (u,v) is an edge in G_L
- 11 $p \leftarrow p \cup \{v\}; u \leftarrow v$



Dinic 算法

```
12      else delete  $u$  and all adjacent edges from  $G_L$ 
13          remove  $u$  from the end of  $p$ .
14          set  $u$  to the last vertex in  $p$  ( $u$  may be  $s$ )
15      if  $u=t$  then
16          let  $c_f(p)$  be the bottleneck capacity along  $p$ .
17          augment the current flow along  $p$  by  $b$ .
18          adjust capacities along  $p$  and delete saturated edges
19          set  $u$  to the last vertex on  $p$  reachable from  $s$  (Note that  $u$ 
    may be  $s$ )
20  compute a new level graph  $G_L$  from  $G_f$ .
```

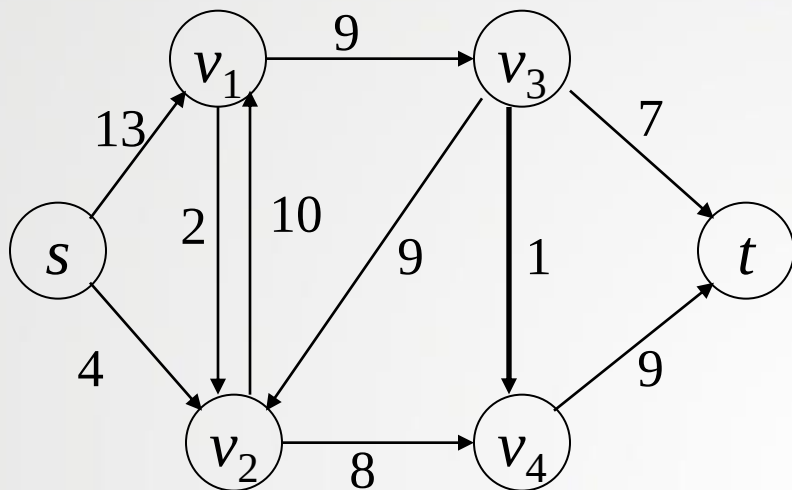


Dinic 算法

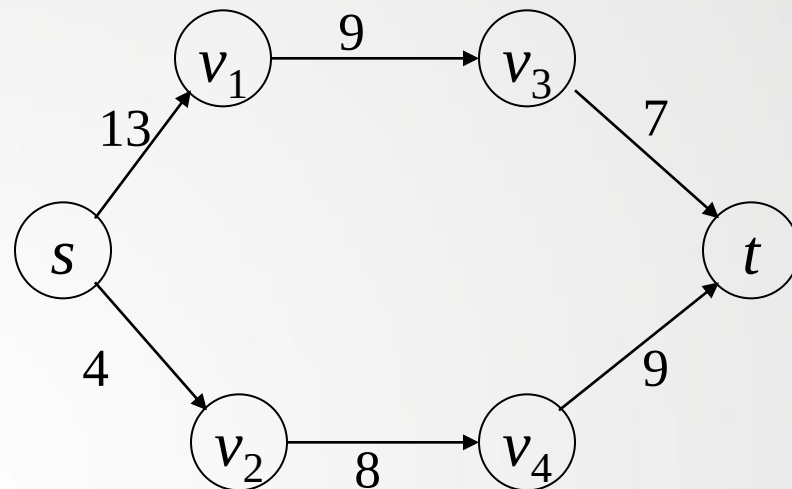
- 定理9.3 给定一个网络 $G=(V,E)$, $\text{Dinic}(G, s, t, c)$ 算法找到最大流的时间为 $O(|E||V|^2)$ 。



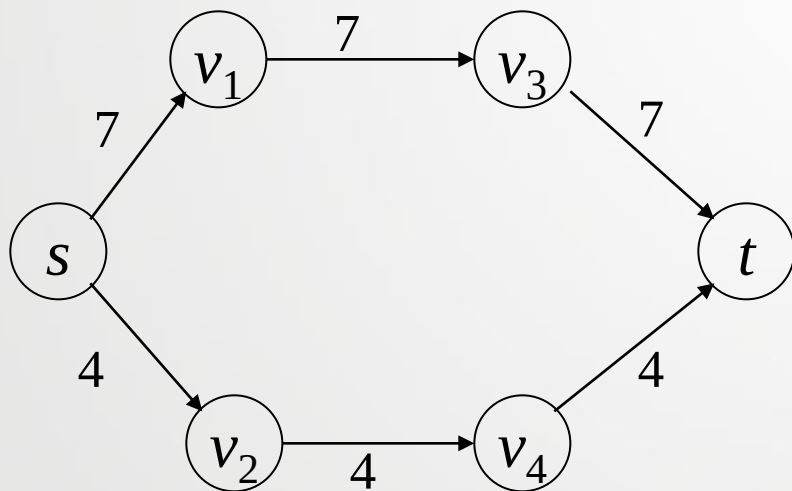
Dinic 算法



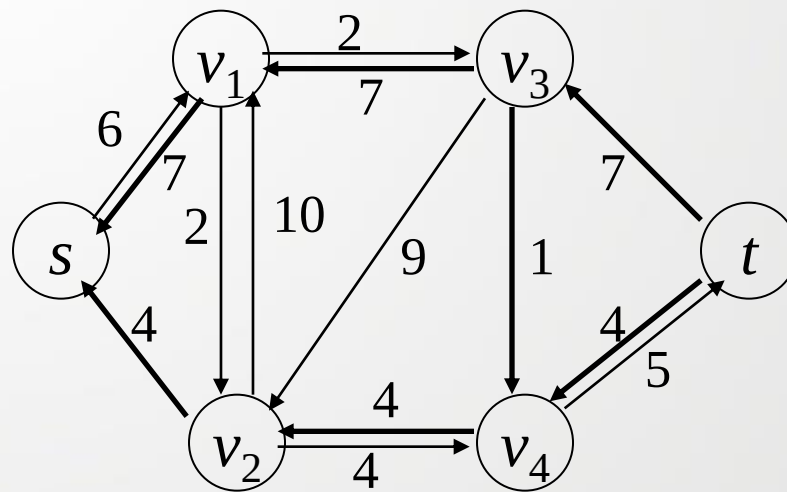
网络 $G=(V,E)$



First level graph G_L



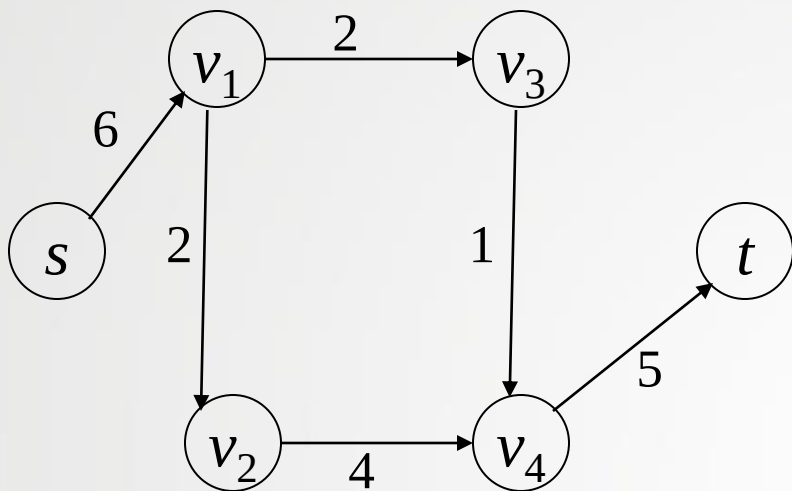
阻塞流



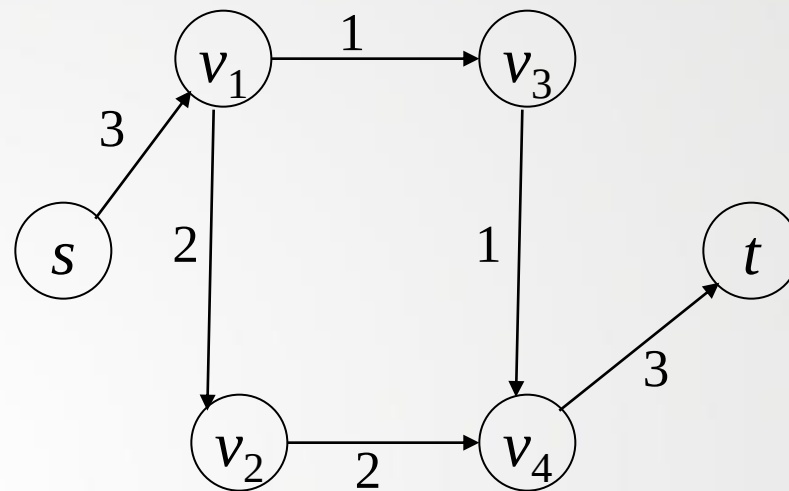
剩余网络



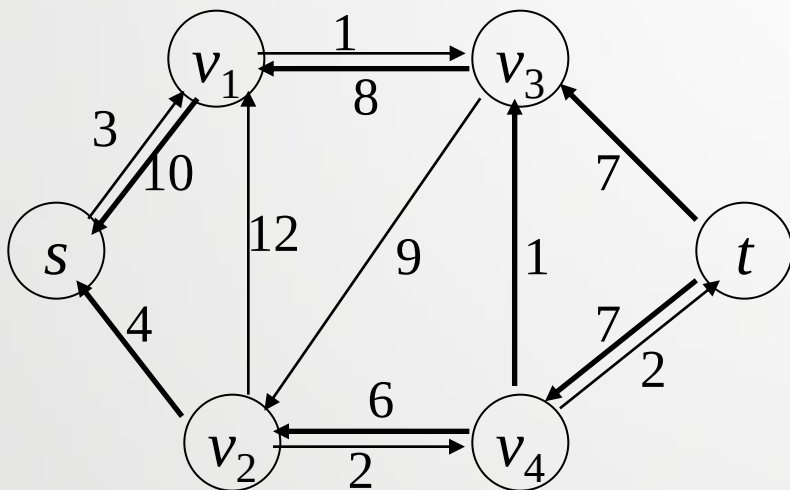
Dinic 算法



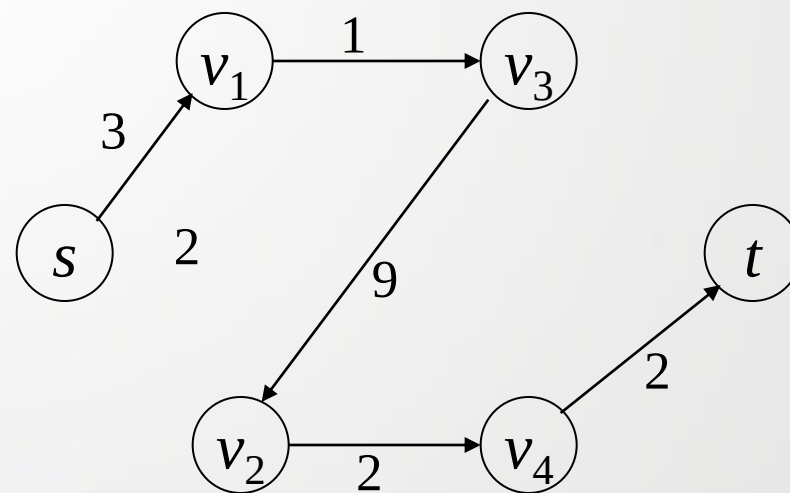
Second level graph G_L



阻塞流



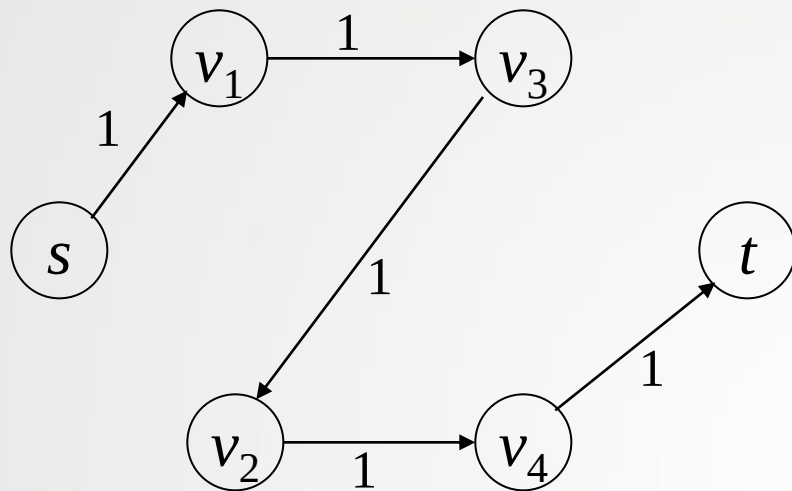
剩余网络



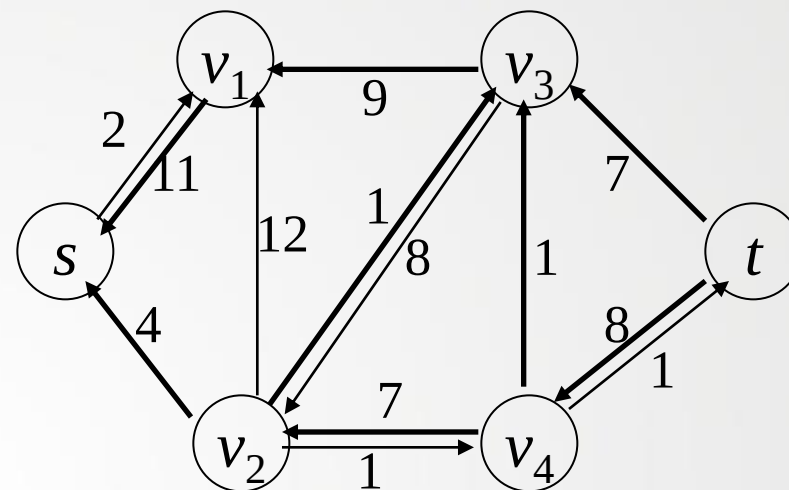
Second level graph G_L



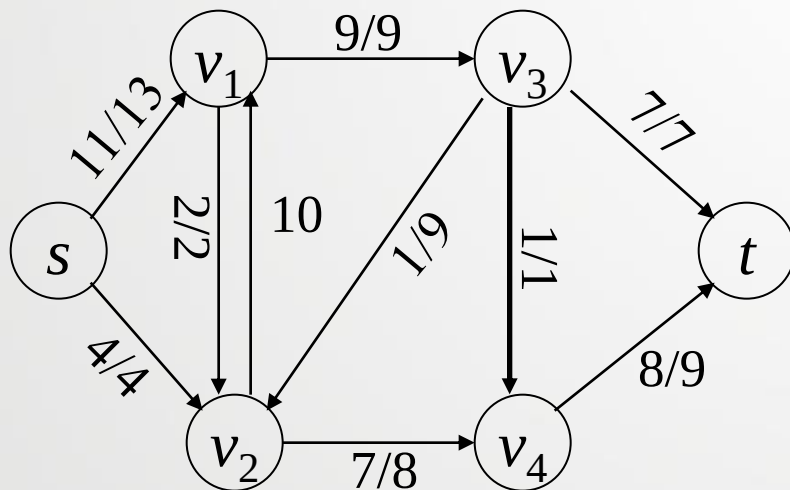
Dinic 算法



阻塞流



剩余网络



New flow network $G=(V,E)$

最后得到:

$$|f| = f(s, V) = 15$$



MPM算法

- **定义9.10** 给定一个网络 $G=(V,E)$ ，对于网络 G 的顶点 v ，我们定义 v 的吞吐量 $C(v)$ 为该顶点入边的总容量和出边的总容量的最小值。即对于 $v \in V - \{s, t\}$,

$$C(v) = \min \left\{ \sum_{u \in V} c(u, v), \sum_{u \in V} c(v, u) \right\}$$

对于源点 s 和汇点 t ，我们分别定义其吞吐量如下

$$C(s) = \sum_{v \in V - \{s\}} c(s, v) \text{ 和 } C(t) = \sum_{v \in V - \{t\}} c(v, t)$$



MPM算法

MPM(G, s, t, c)

- 1 **for** each edge $(u, v) \in E$ **do**
- 2 $G_f \leftarrow G$; find the level graph G_L of G_f
- 3 **while** t is a vertex in G_L **do**
- 4 **while** t is reachable from s in G_L **do**
- 5 find a vertex v of minimum $C(v)$
- 6 push $C(v)$ units of flow from v to t
- 7 pull $C(v)$ units of flow from s to v
- 8 update f , G_L and G_f
- 9 use G_f to compute a new level graph G_L



MPM算法

- 定理9.4 给定一个网络 $G=(V,E)$, $\text{MPM}(G, s, t, c)$ 算法找到最大流的时间为 $O(|V|^3)$ 。

谢谢大家